

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIAS DE LA COMPUTACIÓN CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

GA – EXPOSICIÓN SEGUNDO CORTE: HERRAMIENTAS CASE (BOUML)

ASIGNATURA:

INGENIERÍA DE REQUISITOS

DOCENTE:

ING. GLEISTON GUERRERO ULLOA, PhD.

INTEGRANTES:

CHAVARRIA CUENCA TAHINY MEL, CI: 0943050054, tchavarriac@uteq.edu.ec

MENDOZA PÁRRAGA ANDY JOHEL, CI: 1251401590, amendezap9@uteq.edu.ec, Titular PFC

SANCHEZ CORNEJO GARY ALBERTO, CI: 1208338291, gsanchezc6@uteq.edu.ec

EQUIPO:

H

CURSO/PARALELO:

CUARTO SEMESTRE PARALELO "B"

SISTEMA PFC:

MundiPets (Titular: Andy Mendoza Párraga)

HERRAMIENTA MDD SELECCIONADA:

BOUML

REPOSITORIO GITHUB:

<https://github.com/andymendezap03/PFC-MundiPets-MDD-EquipoH-BOUML.git>

QUEVEDO – LOS RÍOS – ECUADOR

2026 – 2027 PPA

Índice

1. Resumen	2
Resumen	2
2. Marco teórico	2
2.1. Model-Driven Engineering (MDE)	2
2.2. Model-Driven Architecture (MDA)	2
2.3. Model-Driven Development (MDD)	2
2.4. Transformaciones M2M	3
2.5. Transformaciones M2T	3
2.6. Metamodelo	3
2.7. Perfiles UML	3
2.8. Plantilla / template	4
2.9. Forward, reverse y round-trip engineering	4
2.10. Diagramas UML admisibles como origen	4
2.11. Estado del arte	4
3. Justificación de la selección de BOUML	5
3.1. Herramienta seleccionada y versión	5
3.2. Justificación frente a los requisitos del PFC	5
3.3. Comparación con alternativa descartada	5
4. Descripción del subsistema del PFC modelado	6
4.1. Contexto de MundiPets	6
4.2. Módulo seleccionado	6
4.3. Actores	6
4.4. Requisitos funcionales	6
4.5. Alcance del módulo modelado	7
5. Modelo UML de origen	7
5.1. Diagrama de clases	7
5.2. Relaciones UML	9
5.3. Estereotipos	9
5.4. Atributos y operaciones tipificados	9
5.5. Diagramas complementarios	9
5.6. Validación del modelo en BOUML	10
6. Configuración del generador	11
6.1. Plantilla y parámetros	11
6.2. Política de regeneración y decisiones de mapeo	11
7. Proceso de generación	12
7.1. Ejecución y captura	12
7.2. Árbol de proyecto y trazabilidad clase → archivo	12
8. Verificación del código generado	13
8.1. Compilación y ejecución	13
8.2. Correspondencia modelo → código	13
8.3. Limitaciones detectadas	14
9. Cierre del ciclo (roundtrip)	14
9.1. Cambio aplicado al modelo	14
9.2. Evidencia antes/después	15
10. Reparto del trabajo	16
10.1. Trazabilidad de contribuciones técnicas e historial en GitHub	16
10.2. Log de aportaciones individuales (Insights Contributors)	17
11. Conclusiones	17
12. Referencias	18
Anexos	20

1. Resumen

El presente documento evidencia la aplicación práctica del desarrollo guiado por modelos (MDD) para resolver la desincronización existente entre el diseño conceptual y la implementación en el Proyecto de Fin de Carrera (PFC) *MundiPets*. Para mitigar el retrabajo y la pérdida de trazabilidad provocados por la traducción manual de requisitos, se seleccionó la herramienta CASE BOUML 7.11 frente a la alternativa analizada Modelio 5.4.1, debido a su flujo de configuración directo para la ingeniería directa. El modelado se enfocó exclusivamente en el alcance transaccional del subsistema de Gestión de Mascotas e Historial Médico, estructurando diagramas estáticos y de comportamiento bajo restricciones estrictas del metamodelo UML. Como resultado alcanzado, el modelo superó con éxito la auditoría sintáctica interna (*Uml projection Done*), garantizando la consistencia estructural necesaria para generar automáticamente los artefactos físicos de código fuente orientados al backend del sistema. Esta transformación automatizada se configuró utilizando de forma nativa el lenguaje C++, produciendo archivos de cabecera (.h) e implementación (.cpp) completamente limpios, verificables y preparados para la compilación, cerrando el ciclo de trazabilidad propuesto.

2. Marco teórico

2.1. Model-Driven Engineering (MDE)

La Model-Driven Engineering (MDE), o Ingeniería Dirigida por Modelos, constituye un paradigma de desarrollo de software en el que los modelos dejan de considerarse simples bosquejos o diagramas de documentación visual para convertirse en los artefactos centrales de todo el ciclo de vida del sistema [1, 2]. Bajo este enfoque, los modelos se explotan para abstraer las restricciones, reglas de negocio y comportamiento lógico del dominio con un nivel de abstracción muy superior al del código fuente tradicional [3, 4]. A diferencia de los entornos low-code comerciales, MDE se caracteriza por el uso riguroso de lenguajes de modelado estandarizados, el establecimiento formal de metamodelos y el uso de trazas explícitas que aseguran la trazabilidad bidireccional entre los requisitos y la arquitectura de software generada [5, 6].

En el Proyecto de Fin de Curso (PFC) *MundiPets*, este paradigma se operativiza al transformar la especificación de requisitos en un modelo conceptual sólido dentro del entorno operativo de BOUML [7, 8]. Los procesos del dominio clínico, que abarcan desde el control de las hojas sanitarias hasta la gestión de perfiles biológicos de los animales, se modelan mediante estructuras jerárquicas tipadas, asegurando que el diseño resguarde una correspondencia exacta y verificable con las necesidades transaccionales declaradas para la plataforma web [7, 8].

2.2. Model-Driven Architecture (MDA)

La iniciativa Model-Driven Architecture (MDA), promovida por el Object Management Group (OMG), establece un marco de trabajo de ingeniería que aísla las especificaciones funcionales del negocio de las fluctuaciones de las plataformas tecnológicas subyacentes [9]. Para cumplir este propósito, MDA estructura el proceso de desarrollo de software dividiéndolo en tres niveles independientes de abstracción que interactúan mediante transformaciones sucesivas [9]:

- **CIM** (*Computation Independent Model*): Captura las reglas del dominio y las necesidades del negocio de forma puramente conceptual, prescindiendo por completo de consideraciones computacionales o de sistemas.
- **PIM** (*Platform Independent Model*): Expresa el diseño de la arquitectura, las entidades y las interconexiones del sistema de manera lógica, sin vincularse a tecnologías o frameworks concretos.
- **PSM** (*Platform Specific Model*): Vincula el diseño formal (PIM) con los detalles técnicos, tipos primitivos y restricciones de una plataforma de despliegue específica para posibilitar su codificación [10].

A pesar de que la literatura industrial concentra sus esfuerzos en el análisis de la transformación técnica PIM-PSM, la transición del modelo CIM al PIM es clave, ya que es el nexo formal que traslada los requisitos de negocio atómicos hacia el primer modelo de diseño puro e independiente de la plataforma [10].

2.3. Model-Driven Development (MDD)

El Model-Driven Development (MDD), o Desarrollo Dirigido por Modelos, representa la aplicación práctica y automatizada de MDE durante la construcción del software, donde los modelos actúan como las entradas directas para la generación del código [1, 3]. Mediante este enfoque, los generadores automáticos procesan la estructura estática del modelo para instanciar el esqueleto de la aplicación, abatiendo tareas repetitivas de codificación, estandarizando convenciones sintácticas en el equipo de desarrollo y previniendo la inserción de errores estructurales manuales [4, 11]. La idoneidad de emplear herramientas CASE ligeras en este flujo radica en su capacidad de procesar esquemas

lógicos nativos e intercambiables para derivar clases, métodos, firmas funcionales y dependencias sin ambigüedades técnicas [12].

Para el proyecto *MundiPets*, el desarrollo guiado por modelos se concreta al traducir el diagrama de clases del bloque sanitario directamente en la estructura del backend de la aplicación [8]. Las entidades del dominio (*Usuario*, *Mascota*, *HistorialMedico*, *MascotaVacuna* y *EvaluacionVeterinaria*) se configuran con visibilidades y encapsulamientos específicos, permitiendo que la herramienta genere la base limpia del código y forzando al equipo a diferenciar metodológicamente las estructuras generadas por el modelo de la lógica de negocio especializada que se añade a mano [7, 8].

2.4. Transformaciones M2M

Las transformaciones modelo-a-modelo (M2M) toman un modelo de origen y producen un modelo destino, ambos conformes a sus respectivos metamodelos; dentro de MDA, el caso típico es el paso de un PIM a un PSM. El estándar del OMG que rige este tipo de transformación es QVT (*Query/View/Transformation*) v1.3 [13]. No obstante, la evidencia empírica reciente indica que la adopción industrial de lenguajes de transformación dedicados (MTL) como QVT o ATL sigue siendo limitada frente a lo que cabría esperar de un estándar consolidado: el estudio de Höppner et al. [14], basado en 56 entrevistas a investigadores y profesionales, documenta que la falta de tooling maduro y de evidencia sólida sobre sus ventajas frena su uso frente a lenguajes de propósito general.

Esta brecha entre el estándar y la práctica industrial explica un patrón frecuente en las herramientas MDD: muchas no exponen una etapa M2M diferenciada ni un lenguaje de transformación independiente conforme a QVT, sino que absorben esa lógica dentro del propio generador de código, traduciendo el modelo de origen directamente a texto sin materializar un PSM intermedio como artefacto propio. Esto no invalida la distinción conceptual entre M2M y M2T: evidencia, más bien, que en la práctica ambas etapas suelen fusionarse cuando el objetivo final es la generación de código y no la interoperabilidad entre modelos.

2.5. Transformaciones M2T

Las transformaciones modelo-a-texto (M2T) generan artefactos textuales -código fuente, configuración o documentación- a partir de un modelo conforme a un metamodelo; el estándar OMG de referencia es MOFM2T v1.0 [15]. Un ejemplo reciente de esta lógica se encuentra en el trabajo de Jiménez-Navajas et al. [16], quienes implementan transformaciones M2T con el lenguaje EGL (*Epsilon Generation Language*) para producir código Python y Qiskit a partir de modelos UML extendidos, lo que evidencia cómo un motor M2T recorre los elementos del modelo y los inyecta en plantillas parametrizadas para producir texto ejecutable.

Las herramientas MDD implementan esta lógica de dos formas distintas. Por un lado, motores declarativos como Acceleo o EGL exponen la plantilla como un archivo de texto editable que sigue, con distinto grado de fidelidad, la gramática propuesta por MOFM2T. Por otro lado, un grupo amplio de herramientas comerciales integra la generación como un módulo o plug-in compilado que lee las clases, atributos, relaciones y operaciones del modelo y las vuelca en código siguiendo plantillas fijas embebidas en el propio generador, sin exponer una gramática pública ni permitir su edición directa; esta segunda variante limita la personalización a los parámetros que la propia herramienta expone.

2.6. Metamodelo

El metamodelo constituye la capa de abstracción que define las reglas gramaticales y la sintaxis abstracta que rige a un lenguaje de modelado; en el caso de UML, su semántica formal está especificada sobre el estándar MOF (*Meta-Object Facility*) del OMG [17]. Los generadores de código automatizados no interpretan de manera libre o visual los diagramas de un lienzo, sino que ejecutan reglas sintácticas analizando las relaciones lógicas que el metamodelo base garantiza que existen en el árbol del proyecto [18]. La calidad y precisión de este metamodelo restringe directamente la viabilidad de la ingeniería directa, puesto que cualquier ambigüedad u omisión de tipos degrada la estructura de los artefactos generados [19]. BOUML procesa recursivamente esta sintaxis abstracta para traducir de forma exacta las relaciones de generalización, agregación y composición hacia punteros, vectores y herencias nativas de C++ [20].

2.7. Perfiles UML

Los perfiles UML representan el mecanismo estándar diseñado por el OMG para habilitar la extensión ligera del metamodelo, adaptando el vocabulario y comportamiento genérico de UML a plataformas tecnológicas o dominios de negocio específicos sin modificar la especificación base del lenguaje [17]. Este mecanismo opera inyectando estereotipos, restricciones semánticas y valores etiquetados que añaden una carga semántica especializada, la cual es

leída por los motores de ingeniería directa para decidir qué tipo de arquitectura física inyectar en el código destino [21, 22]. Para el subsistema clínico de *MundiPets*, se aplicó el estereotipo «Entity» a las clases de dominio encargadas de la persistencia (*Mascota*, *HistorialMedico*, *Vacuna*) y «Actor» a las clases de control e interacción de usuarios, permitiendo al generador discriminar la lógica de los archivos de salida [7, 20].

2.8. Plantilla / template

Una plantilla o *template* es el fragmento parametrizado que la herramienta MDD instancia por cada elemento del modelo para producir el código correspondiente. Este mecanismo puede implementarse como archivo de texto editable, ligado a una gramática de tipo MOFM2T, o puede empaquetarse como un componente compilado que aplica la definición del modelo sin exponerla como texto. Paolone et al. [23] documentan este segundo caso con xGenerator, una plataforma MDA propietaria que genera aplicaciones Java MVC completas a partir de un modelo UML sin exponer la plantilla al desarrollador, tratándola como una caja negra cuya salida solo puede evaluarse por la calidad del código resultante y no por inspección directa de la plantilla.

Un mecanismo de parametrización habitual en este tipo de generadores es el bloque protegido: toda operación cuyo cuerpo se delimita mediante marcadores reservados conserva, en regeneraciones sucesivas, el contenido que el desarrollador haya escrito manualmente, evitando que una nueva ejecución del generador sobrescriba la lógica de negocio ya implementada. Este mecanismo es coherente con la lógica M2T y con el estándar MOFM2T [15], y constituye la base técnica que permite un ciclo de *roundtrip* controlado: el modelo puede regenerarse repetidamente sin perder el trabajo manual, siempre que este se mantenga dentro de los delimitadores correspondientes.

2.9. Forward, reverse y round-trip engineering

La ingeniería de software bidireccional abarca tres procesos de transformación complementarios orientados a mantener la consistencia entre el diseño y los archivos físicos: el *forward engineering* (ingeniería directa) automatiza la traducción de modelos abstractos a código fuente; el *reverse engineering* analiza estáticamente los archivos fuente en disco para recuperar su estructura y plasmarla en diagramas UML; y el *round-trip engineering* unifica ambos sentidos, gestionando la sincronización interactiva cuando cualquiera de los dos artefactos experimenta cambios evolutivos [1, 3, 24]. Mantener esta paridad en entornos industriales es uno de los mayores desafíos metodológicos debido a las modificaciones concurrentes y a la inserción de código manual [25]. En *MundiPets*, el ciclo se cierra aplicando un cambio controlado en el modelo de clases, regenerando las firmas en C++ y verificando físicamente que BOUML respete el código humano embebido en sus bloques protegidos [8].

2.10. Diagramas UML admisibles como origen

El diagrama de clases representa el artefacto nuclear e imprescindible para la generación estructural de código fuente dentro del paradigma MDD, relegando a vistas de comportamiento o interacción a un rol estrictamente complementario [1]. Esta jerarquía teórica se convalida plenamente con la evidencia empírica industrial: encuestas globales reflejan que el diagrama de clases es la primera opción elegida unánimemente por los desarrolladores para modelar la estructura funcional (71 %) y la arquitectura de datos (85 %) de los sistemas complejos, demostrando que centrar la ingeniería directa en esta vista estática responde a la práctica dominante del desarrollo corporativo [26]. En concordancia con este alcance, el módulo de gestión sanitaria de *MundiPets* se modeló explotando esta perspectiva estructural de tipos y herencias, integrándose de forma nativa con el diagrama de casos de uso para delimitar las fronteras de interacción de los actores en BOUML [8].

2.11. Estado del arte

En la actualidad, la Ingeniería Dirigida por Modelos ha evolucionado hacia la confluencia con lenguajes específicos de dominio (DSL), plataformas low-code avanzadas y flujos de automatización integrados en arquitecturas DevOps [27, 28]. A pesar de estos avances, persisten desafíos de investigación abiertos en torno a la escalabilidad de las transformaciones M2M complejos, la preservación a largo plazo de mapas de trazabilidad multidimensionales y la sincronización continua entre el diseño gráfico y la implementación física [25, 29]. En este panorama, herramientas CASE ligeras y robustas como BOUML 7.11 demuestran su vigencia al proveer un entorno óptimo para la ingeniería estructural, abstrayendo la complejidad de la codificación repetitiva, garantizando compilaciones nativas limpias y certificando un nexo auditable entre los requisitos atómicos de la ingeniería de requisitos y el software final en C++ [8, 30].

3. Justificación de la selección de BOUML

3.1. Herramienta seleccionada y versión

La herramienta seleccionada para desarrollar la demostración de Ingeniería Dirigida por Modelos (MDD) aplicada al PFC *MundiPets* es BOUML 7.11, parche 4 [30]. Esta versión fue escogida por ofrecer un entorno UML optimizado y de alto rendimiento orientado al modelado estructural, la generación automática de código, la ingeniería inversa y el *round-trip engineering* [20]. BOUML es una herramienta UML 2 de uso gratuito y multiplataforma, con soporte nativo para diferentes lenguajes de programación estructurados y orientados a objetos, entre los cuales se destaca C++ debido a su eficiencia en la administración física de la memoria del backend [20, 30].

Para la ejecución práctica del proyecto, se configuró C++ como el lenguaje objetivo central. BOUML se empleará para construir el diagrama de clases y de casos de uso del subsistema sanitario, transformando el diseño conceptual directo en archivos físicos de implementación base (.h y .cpp) [20]. El alcance operativo del entorno CASE se concentrará estrictamente en los elementos derivados de forma directa del metamodelo UML, delegando la capa de persistencia SQL, las interfaces gráficas y las reglas del negocio específicas a una fase complementaria de desarrollo manual [7]. La versión seleccionada garantiza un entorno libre de errores sintácticos y reproducible, permitiendo que cualquier integrante del equipo pueda regenerar el código fuentes con las mismas marcas arquitectónicas en cualquier estación de trabajo [8].

3.2. Justificación frente a los requisitos del PFC

La selección de BOUML se justificó analizando las restricciones funcionales de *MundiPets*, la arquitectura modular planteada en la ERS y la obligatoriedad de demostrar la trazabilidad modelo-código en la evaluación académica [7, 8].

- **Correspondencia con el dominio modelado:** El subsistema clínico puede estructurarse limpiamente mediante clases del negocio como *Usuario*, *Mascota*, *HistorialMedico*, *Vacuna*, *MascotaVacuna* y *EvaluacionVeterinaria* [7]. BOUML permite tipificar atributos primitivos, firmas de operaciones públicos/protegidos y cardinalidades complejas, resolviendo la lógica estática del dominio sin ambigüedades técnicas [20].
- **Generación eficiente de código backend:** La herramienta automatiza la traducción estructural, reduciendo la escritura manual redundante y garantizando que las asociaciones (como la composición fuerte mascota-historial) se transformen en punteros y vectores fuertemente tipados bajo el estándar de C++ [20]. Esto disminuye el retrabajo metodológico y formaliza las bases de la aplicación [12].
- **Arquitectura modular y encapsulamiento:** La ERS de *MundiPets* exige un sistema desacoplado para integrar dinámicamente futuros componentes como módulos de inteligencia artificial o pasarelas de adopción [7]. En BOUML, las clases se organizan mediante paquetes lógicos y vistas conceptuales separadas del Browser, forzando la separación inicial de responsabilidades y un control estricto de visibilidades [20].
- **Trazabilidad en el cierre del ciclo (Round-trip):** La herramienta cumple de manera nativa con la restricción académica de inyectar cambios controlados en el diagrama (como la inserción de nuevos atributos o métodos), regenerar los archivos físicos y verificar visualmente que las firmas se actualicen en los fuentes sin corromper la lógica programada a mano en sus bloques protegidos [8, 20].
- **Licencia y curva de aprendizaje:** Al ser un software de uso gratuito y consumo ligero de recursos físicos, optimiza el trabajo concurrente del equipo sin requerir inversiones comerciales [30]. Su interfaz compacta permite focalizar la atención en la consistencia de los diagramas, la parametrización de plantillas y el éxito de la compilación directa [20].

3.3. Comparación con alternativa descartada

La alternativa de código abierto analizada por el equipo fue Modelio 5.4.1, la cual admite el modelado de múltiples lenguajes y estándares de arquitectura como BPMN, ArchiMate y SysML mediante la adición de extensiones modulares [31]. Si bien Modelio posee robustez corporativa, fue descartada formalmente debido a que la sobrecarga en la configuración de sus módulos de generación Java/C++ y su pesada interfaz gráfica excedían el alcance inmediato exigido para la demostración transaccional de *MundiPets* [8]. Para los objetivos académicos planteados, BOUML ofrece un flujo de trabajo significativamente más directo, un motor de ingeniería directa integrado de forma nativa en la raíz del proyecto y una menor fricción técnica para validar la consistencia del código generado [8, 20].

Tabla 1: Comparación técnica entre BOUML y la alternativa descartada Modelio.

Criterio	BOUML 7.11 (Parche 4)	Modelio 5.4.1	Decisión para MundiPets
Licencia y Costo	Uso gratuito; no requiere licencias comerciales [30].	Código abierto sin costos de activación [31].	Ambos cumplen gratuidad académica [8].
Generación C++	Motor nativo integrado; mapeo directo a pares .h y .cpp [20].	Requiere instalación de módulos externos.	Flujo directo y limpio hacia C++ en BOUML [20].
Alcance Funcional	Entorno concentrado en estándar UML e ingeniería bidireccional [20].	Ecosistema extenso que abarca BPMN y ArchiMate [31].	Capacidades extras de Modelio no requeridas [8].
Configuración	Parametrización desde la raíz del proyecto [20].	Exige activar módulos por separado [31].	BOUML reduce componentes críticos a configurar [8].
Evolución y Ciclo	Prioriza estabilidad del metamodelo y bloques protegidos [20].	Orientado a modelado empresarial continuo [31].	BOUML ofrece relación más directa para verificación [8].

4. Descripción del subsistema del PFC modelado

4.1. Contexto de MundiPets

MundiPets se constituye como una plataforma digital integral orientada a facilitar y organizar los procesos de adopción y cruce responsable de mascotas, respondiendo a la necesidad de contar con un canal centralizado, transparente y seguro. El sistema actúa como un nodo de conexión entre diversos actores clave, incluyendo propietarios de mascotas, adoptantes, interesados en cruce, refugios de animales y veterinarias.

A través de esta plataforma, se busca centralizar y digitalizar la gestión de perfiles biológicos de los animales, el almacenamiento de historiales clínicos, la publicación de anuncios y la verificación de certificados. De este modo, MundiPets organiza de manera eficiente la interacción entre los usuarios que desean dar o recibir una mascota en custodia, aquellos que buscan un emparejamiento biológico controlado para sus ejemplares, y los profesionales veterinarios encargados de avalar la veracidad sanitaria de la información registrada.

4.2. Módulo seleccionado

El componente del sistema seleccionado para trabajar en la herramienta corresponde al subsistema de Gestión de Mascotas e Historial Médico de MundiPets. Este bloque operativo es fundamental, ya que se encarga de administrar toda la información básica, clínica y de control de los animales antes de habilitar otros procesos dentro de la plataforma.

De acuerdo con el diagrama del sistema, el módulo está conformado por un conjunto interconectado de clases que organizan el dominio: la clase base *Usuario* junto con sus herencias específicas (*Propietario* y *Veterinario*), la entidad central *Mascota*, el *HistorialMedico* (vinculado mediante una relación de composición rígida), el catálogo de *Vacuna*, la clase asociativa *MascotaVacuna* (que resuelve el carné sanitario) y la entidad *EvaluacionVeterinaria* para registrar las revisiones de salud. A través de estas clases, el subsistema permite registrar los datos generales de los animales, almacenar antecedentes clínicos, controlar las dosis de vacunas aplicadas y gestionar las evaluaciones profesionales correspondientes.

4.3. Actores

Los actores que interactúan de forma directa con el subsistema seleccionado se definen a partir de los roles representados en la jerarquía del modelo UML:

- **Propietario:** Es el usuario que registra y administra a los animales dentro de la plataforma. Sus acciones principales incluyen ingresar la información general de la mascota, actualizar sus datos básicos, registrar antecedentes médicos iniciales y gestionar sus publicaciones en el sistema.
- **Veterinario:** Es el actor profesional encargado de auditar la información de salud. Sus funciones dentro de este módulo abarcan la validación de los historiales clínicos, la carga y revisión de documentos oficiales de respaldo, y el registro de las evaluaciones veterinarias correspondientes.

4.4. Requisitos funcionales

El alcance del subsistema modelado comprende las operaciones de registro, actualización, consulta y validación profesional de la información biológica y clínica de los animales. Los requisitos funcionales específicos que rigen este módulo se detallan a continuación:

Tabla 2: Requisitos funcionales del subsistema de Gestión de Mascotas e Historial Médico.

ID	Requisito	Descripción
RF-01	Registrar mascota	El sistema deberá permitir al propietario registrar una mascota ingresando su información general, incluyendo nombre, especie, raza, sexo, edad, estado reproductivo y fotografías para su identificación dentro de la plataforma.
RF-02	Gestionar historial médico	El sistema deberá permitir registrar y actualizar el historial médico de una mascota, incluyendo vacunas, desparasitaciones, enfermedades, alergias, cirugías, tratamientos y controles veterinarios.
RF-03	Consultar perfil completo	El sistema deberá permitir visualizar el perfil completo de una mascota mostrando su información general, historial médico autorizado, fotografías, comportamiento, vacunas, edad, raza para procesos de adopción o cruce.
RF-09	Validar información médica	El sistema deberá permitir que una veterinaria autorizada valide la información médica registrada de una mascota, indicando que los datos han sido verificados por un profesional.
RF-14	Gestionar carnet de vacunación	El sistema deberá permitir registrar, actualizar y consultar el carnet de vacunación de cada mascota, mostrando el historial de vacunas aplicadas, fechas, próximas dosis y su estado de vigencia.
RF-16	Gestionar antecedentes genéticos	El sistema deberá permitir registrar y consultar los antecedentes genéticos y el grado de parentesco de una mascota, incluyendo información sobre enfermedades hereditarias y linaje para apoyar los procesos de cruce responsable y la evaluación de compatibilidad.

La documentación detallada con la caracterización completa de los ocho atributos de gestión y la especificación atómica de cada uno de estos requisitos funcionales se encuentra recopilada de manera íntegra en el **Anexo A**.

4.5. Alcance del módulo modelado

El alcance del trabajo con la herramienta se enfoca exclusivamente en la definición de la estructura estática del subsistema de mascotas e historial médico, estableciendo los límites de lo que se incluye y lo que se excluye en este proceso:

Elementos incluidos dentro del alcance:

- Definición visual de las clases involucradas con sus respectivos nombres, atributos tipificados (como *string* o *long*) y firmas de métodos públicos o privados.
- Representación de las relaciones UML del dominio, incluyendo la generalización (herencia), la composición fuerte entre la mascota y su historial, y las asociaciones con multiplicidades.
- Modelado de la estructura de las relaciones complejas mediante el uso de clases asociativas dedicadas, como es el caso del control del carné de vacunación.

Elementos excluidos del alcance:

- El desarrollo de la lógica interna o programación detallada del cuerpo de los métodos y funciones del negocio.
- La configuración física de esquemas, tablas o conexiones de bases de datos.
- El diseño visual o maquetado de las pantallas de la interfaz gráfica del usuario.

5. Modelo UML de origen

En esta sección se detalla la estructuración, las características técnicas y las restricciones semánticas del modelo UML desarrollado directamente dentro del entorno operativo de BOUML. Este modelo consolida la lógica del negocio para el subsistema de Gestión de Mascotas e Historial Médico de MundiPets, sirviendo como la fuente de datos autoritativa y estructurada para el proceso posterior de ingeniería directa y generación automática de código fuente.

5.1. Diagrama de clases

El diagrama de clases estructural del subsistema sanitario se construyó y normalizó de manera nativa en BOUML, adaptando las especificaciones del dominio a la sintaxis del metamodelo. En la Figura 1 se expone el lienzo de trabajo final, el cual integra la totalidad de los compartimentos de atributos, operaciones y visibilidades requeridos para la correcta compilación del sistema.

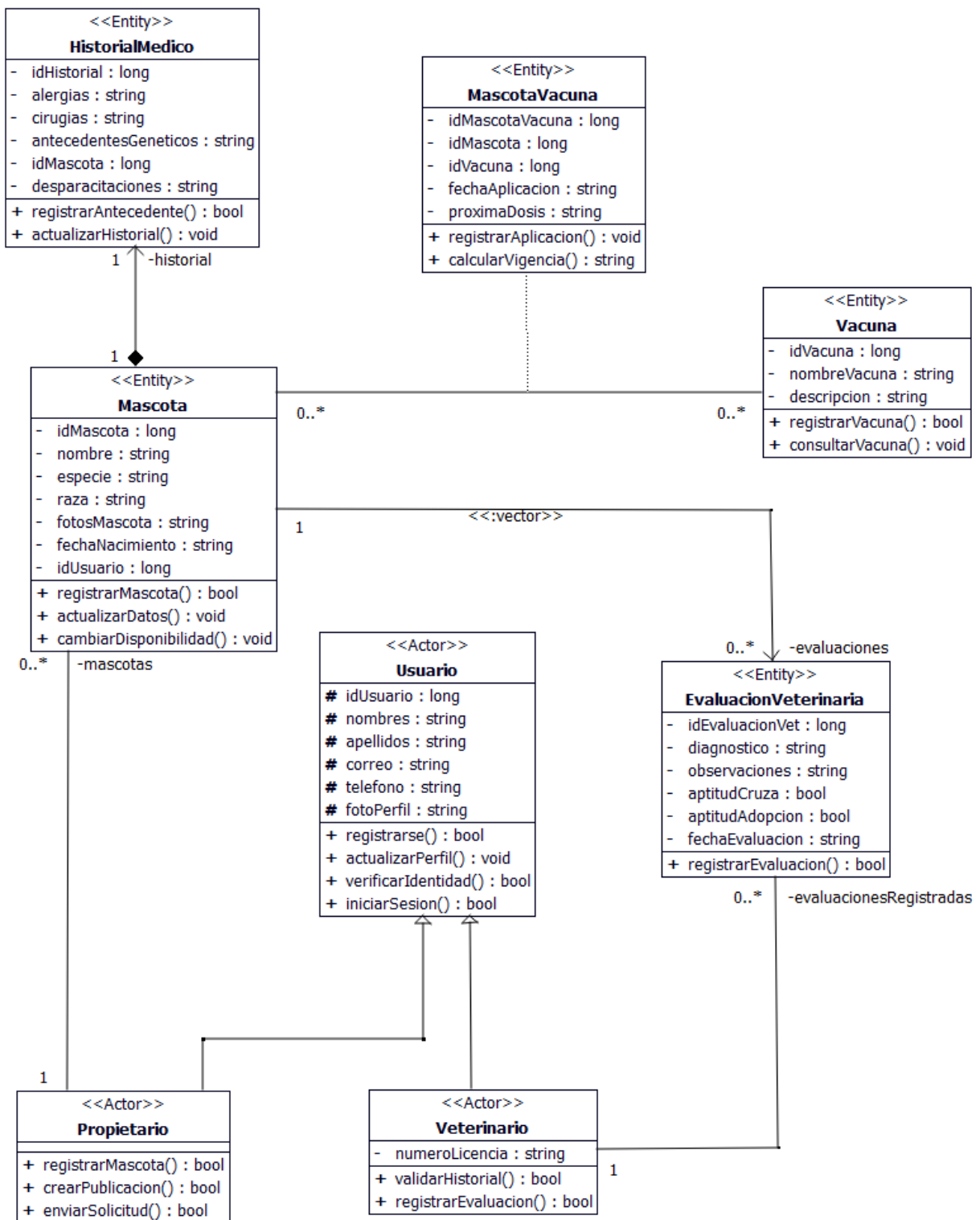


Figura 1: Diagrama de clases del subsistema sanitario refinado en el entorno de BOUML.

5.2. Relaciones UML

Las conexiones que trazamos en el lienzo de BOUML dictan las reglas del negocio y definen cómo se van a comportar los punteros y contenedores en el código de C++. En la Tabla 3 desglosamos detalladamente qué significa cada multiplicidad y tipo de enlazado en nuestro diseño.

Tabla 3: Especificación de relaciones y multiplicidades del modelo en BOUML.

Clases Involucradas	Tipo de Relación	Descripción de la Restricción UML
<i>Usuario</i> ← <i>Propietario</i>	Generalización	Herencia pura. La clase <i>Propietario</i> hereda y especializa los atributos del <i>Usuario</i> base.
<i>Usuario</i> ← <i>Veterinario</i>	Generalización	Herencia pura. La clase <i>Veterinario</i> extiende al <i>Usuario</i> común, añadiendo su licencia profesional.
<i>Mascota</i> ♦ → <i>HistorialMedico</i>	Composición	Relación de existencia fuerte (1:1). El <i>HistorialMedico</i> depende totalmente de que exista la <i>Mascota</i> .
<i>Propietario</i> (1) ↔ <i>Mascota</i> (0..*)	Asociación Simple	Navegabilidad que define la propiedad. Un dueño administra de cero a muchas mascotas en su cuenta.
<i>Mascota</i> (*) ↔ <i>Vacuna</i> (*)	Clase Asociativa	Relación muchos a muchos (M:N) del estándar UML, mapeando vacunas aplicadas del catálogo.
<i>Mascota</i> (1) ↔ <i>MascotaVacuna</i> (*)	Vínculo Instancia	Conexión estructural que une a una mascota específica con las dosis de su carné sanitario.
<i>Vacuna</i> (1) ↔ <i>MascotaVacuna</i> (*)	Vínculo Catálogo	Conexión que asocia el registro de la dosis aplicada con las especificaciones de la vacuna base.
<i>Mascota</i> (1) → <i>EvaluacionVeterinaria</i> (0..*)	Asociación Unidireccional	Define el flujo de revisiones. Una mascota acumula un historial de evaluaciones clínicas mediante un vector.
<i>Veterinario</i> (1) ↔ <i>EvaluacionVeterinaria</i> (0..*)	Asociación Simple	Vincula la responsabilidad clínica. Un veterinario es el encargado de registrar múltiples evaluaciones.

5.3. Estereotipos

Para que el diagrama no sea solo un montón de cajas y el generador de BOUML entienda qué función cumple cada clase en la vida real, les asignamos marcas especiales llamadas *estereotipos*. En el diseño del sistema usamos dos tipos principales que nos ayudan a separar los roles de los usuarios de los datos puros del negocio:

- **«Actor»:** Identifica las clases que representan a personas o roles que van a interactuar directamente con el sistema: la clase base *Usuario* y sus herencias directas *Propietario* y *Veterinario*.
- **«Entity»:** Aplicado a todas las clases encargadas de almacenar y hacer persistir la información clave de la clínica veterinaria: *Mascota*, *HistorialMedico*, *Vacuna*, la clase puente *MascotaVacuna* y *EvaluacionVeterinaria*.

5.4. Atributos y operaciones tipificados

Para que la traducción del diagrama al código de C++ sea exacta y no genere errores al compilar, nos aseguramos de que cada atributo y método en BOUML tenga su tipo de dato bien definido. No dejamos ninguna variable al aire, sino que mapeamos todo según las necesidades reales del software:

- **Identificadores únicos (long):** Todas las llaves primarias del sistema (como *idMascota*, *idHistorial* o *idUsuario*) se configuraron como de tipo `long` para garantizar escalabilidad.
- **Cadenas de texto (string):** Las variables destinadas a guardar palabras (nombres, correos, razas) se tipificaron como `string`. Se ocultó el prefijo estándar `std::` en el lienzo gráfico para mantenerlo limpio.
- **Métodos y operaciones (bool / void):** Las funciones que realizan controles o validaciones devuelven un tipo lógico `bool` para conocer su éxito, mientras que las destinadas únicamente a actualizar datos se configuraron con tipo de retorno `void`.

5.5. Diagramas complementarios

Para complementar la perspectiva estática que nos dio el diagrama de clases, es fundamental analizar el comportamiento del sistema desde el punto de vista del usuario. Mientras que las clases definen la estructura interna, las variables y los métodos del código, los diagramas de comportamiento nos permiten entender el alcance funcional del software y qué acciones reales pueden ejecutar los actores sobre el sistema.

Para este propósito, construimos un Diagrama de Casos de Uso de manera centralizada y nativa dentro del entorno de BOUML, enfocándonos exclusivamente en el alcance transaccional del subsistema de Gestión de Mascotas e Historial Médico. En este modelo se delimitan las fronteras del módulo (representadas por el recuadro del límite del sistema) y se detalla cómo los actores disparan los flujos principales:

- **Propietario:** Interactúa directamente con el subsistema activando los casos de uso destinados al registro de sus mascotas, la configuración de la privacidad médica de los datos y la consulta del perfil médico. Asimismo,

maneja el registro del historial médico, flujo que cuenta con una relación de extensión («extend») hacia el caso de uso *Cargar documento veterinario*, indicando que el usuario tiene la opción opcional de adjuntar archivos de soporte durante este proceso.

- **Veterinario:** Domina el flujo técnico e institucional de la clínica, teniendo acceso a la consulta del perfil médico de los pacientes y siendo el único actor encargado de disparar el caso de uso para validar la información médica en el sistema.

El desarrollo de este diagrama de comportamiento dentro de la misma herramienta de origen garantiza una coherencia absoluta en nuestro informe, demostrando que cada caso de uso del negocio cuenta con una correspondencia directa en una clase, un método y un atributo tipificado listo para soportar su ejecución en el código real de C++.

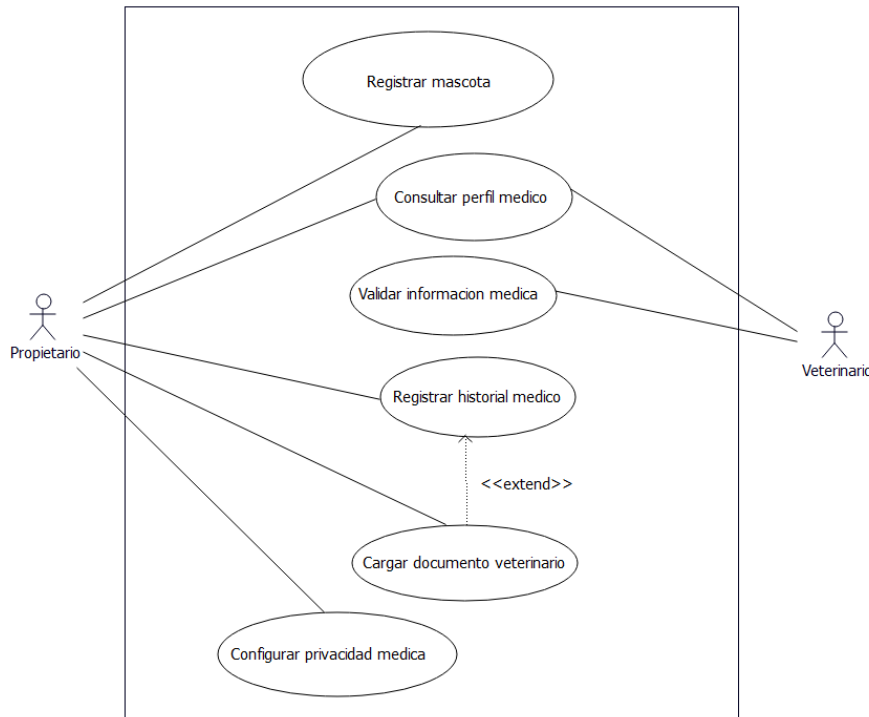


Figura 2: Diagrama de casos de uso del subsistema sanitario desarrollado nativamente en BOUML.

5.6. Validación del modelo en BOUML

Antes de dar el paso definitivo hacia la ingeniería directa y generar el código fuente en C++, sometimos todo el diseño del subsistema a la herramienta de auditoría e inspección interna de BOUML. A diferencia de otros programas que son simples herramientas de dibujo genérico, BOUML cuenta con un analizador sintáctico que comprueba activamente que se cumplan las restricciones reales del metamodelo UML.

Para realizar esta verificación, ejecutamos el control de consistencia jerárquica sobre el árbol del proyecto mediante la herramienta *Uml projection*, monitoreando su comportamiento a través de la consola de trazas. El diagnóstico determinó que nuestro modelo está completamente limpio y libre de errores estructurales, finalizando con un estado exitoso y sin advertencias, tal como se evidencia en la Figura 3.

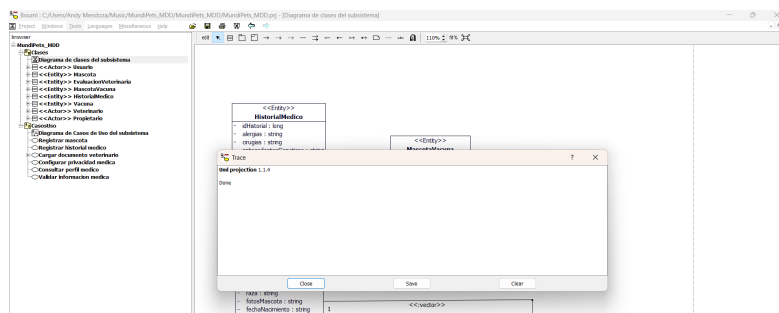


Figura 3: Consola de trazas de BOUML confirmando la validación exitosa del proyecto (Uml projection Done).

Durante este proceso automatizado, el entorno validó los siguientes puntos clave del diseño:

- **Consistencia en las relaciones:** Se comprobó que todas las asociaciones, herencias, composiciones y la clase asociativa interconectan extremos bien definidos, con multiplicidades válidas que el generador puede traducir sin ambigüedades.
- **Integridad de tipos:** Se verificó que ninguna variable o método quedara con tipos de datos "huérfanos," indefinidos, asegurando que los primitivos (`string`, `long`, `bool`, `void`) estén correctamente mapeados.
- **Validación de herencia y visibilidades:** El analizador confirmó que la jerarquía de generalización desde la clase base *Usuario* hacia *Propietario* y *Veterinario* respeta las reglas de encapsulamiento y el acceso a los atributos protegidos (`#`).

6. Configuración del generador

6.1. Plantilla y parámetros

Para transformar el diagrama de clases validado en la Sección 4 en artefactos de implementación, se utilizó el generador C++ integrado de BOUML (versión 7.11 patch 4). En la pestaña *Directory* del cuadro de diálogo *Generation settings*, se estableció el directorio raíz de generación C++ (*C++ root dir*) apuntando a la carpeta generado/ dentro del proyecto. En la pestaña *C++[1]* se confirmaron los parámetros base: extensión de cabecera h y de implementación cpp; directivas `#include` sin ruta; e indentación de 2 espacios para miembros y clases amigas, y 4 espacios para clases anidadas, junto con las guardas de inclusión automáticas (`#ifndef/#define/#endif`) y las plantillas de cabecera e implementación.

Adicionalmente, la pestaña *C++[5]* reveló que BOUML resuelve los tipos contenedores externos (`vector`, `list`, `map`, `string`) mediante una tabla de correspondencia que agrega automáticamente su directiva `#include` correspondiente junto con `using namespace std;`, por lo que el código generado referencia estos tipos sin el prefijo `std::` explícito. La Figura 4 documenta esta configuración.

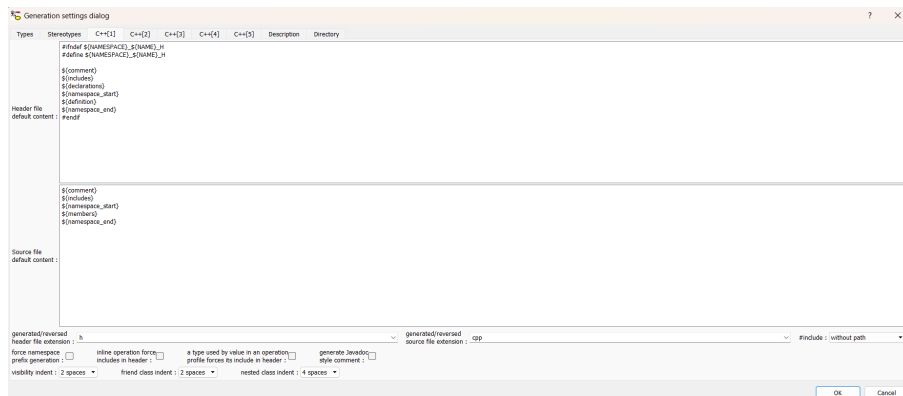


Figura 4: Configuración del generador C++ en BOUML: extensiones de archivo (C++[1]) y resolución de tipos externos con `using namespace std;` (C++[5]).

6.2. Política de regeneración y decisiones de mapeo

La pestaña *C++[3]* del generador confirmó las plantillas exactas que BOUML aplica según el tipo de relación: un atributo simple se declara como `$type $name$value;`, mientras que las relaciones de asociación, agregación y composición con multiplicidad múltiple (*) se declaran anteponiendo un asterisco al nombre (`$type * $name$value;`), es decir, mediante un puntero simple y no mediante un contenedor STL por defecto. Se evaluó la posibilidad de sustituir esta plantilla nativa por el tipo contenedor `vector` definido en la pestaña *C++[5]* (que ya incluye su `#include <vector>` y `using namespace std;` correspondientes) para las asociaciones con multiplicidad múltiple, como *Propietario* (1) ↔ *Mascota* (0..*). Sin embargo, para mantener fiel el criterio de trazabilidad directa con la plantilla auditada por BOUML, el equipo decidió conservar la plantilla nativa de puntero simple sin modificarla; en consecuencia, en el código final generado **todas las asociaciones con multiplicidad múltiple se representan mediante un puntero simple** (`Mascota * mascotas;` en *Propietario.h*, `EvaluacionVeterinaria * evaluaciones;` en *Mascota.h*, etc.), y no mediante contenedores `vector`. Esta decisión implica una limitación funcional real: un puntero simple solo puede referenciar un elemento a la vez, por lo que la navegación 1-N declarada en el modelo no queda completamente representada en el código físico; esta discrepancia se documenta como limitación en la Sección 7.3. Las operaciones de acceso (*getters/setters*) se generaron siguiendo la plantilla de la pestaña *C++[4]*: los métodos `get_$name` se generan públicos, `inline` y `const`, mientras que los `set_$name` se

generan públicos sin modificadores adicionales; sin embargo, BOUML no genera estas operaciones automáticamente por defecto para los atributos del modelo (ver limitaciones, Sección 7.3), por lo que no se aplicaron sobre el código final entregado.

7. Proceso de generación

7.1. Ejecución y captura

La generación de código se ejecutó mediante la opción *Generate* → *C++* del menú contextual sobre el paquete raíz *modelo*, una vez confirmada la configuración descrita en la Sección 5.1. La consola de trazas de BOUML (*Trace*) reportó el uso del **C++ generator release 5.4** y confirmó el resultado *Generation done: 0 warnings, 0 errors*, evidenciando que las ocho clases del modelo (*Usuario*, *Propietario*, *Veterinario*, *Mascota*, *HistorialMedico*, *Vacuna*, *MascotaVacuna* y *EvaluacionVeterinaria*) se procesaron correctamente, produciendo dieciséis archivos —un par *.h/.cpp* por cada clase— sin errores ni advertencias, tal como se evidencia en la Figura 5. Dado el tamaño reducido del subsistema modelado, la generación se completó de forma prácticamente instantánea (menos de un segundo), sin que la consola de trazas reporte un cronómetro explícito para esta operación.

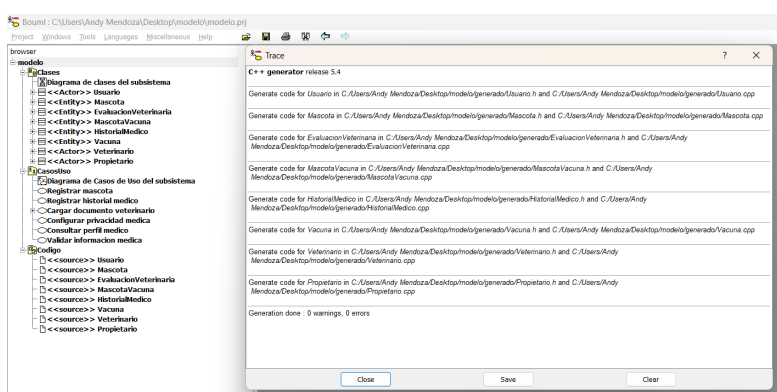


Figura 5: Log de BOUML confirmando la generación exitosa del código C++

7.2. Árbol de proyecto y trazabilidad clase → archivo

Una vez confirmada la generación exitosa, se verificó el contenido de la carpeta configurada como directorio raíz C++ (generado/), constatando la presencia de dieciséis archivos con una misma marca de tiempo, correspondientes a los ocho pares *.h/.cpp* de las clases del modelo. La Tabla 4 y la Figura 6 documentan esta correspondencia.

Tabla 4: Trazabilidad entre clases del modelo UML y archivos C++ generados.

Clase UML	Archivos generados
Usuario	Usuario.h / Usuario.cpp
Propietario	Propietario.h / Propietario.cpp
Veterinario	Veterinario.h / Veterinario.cpp
Mascota	Mascota.h / Mascota.cpp
HistorialMedico	HistorialMedico.h / HistorialMedico.cpp
Vacuna	Vacuna.h / Vacuna.cpp
MascotaVacuna	MascotaVacuna.h / MascotaVacuna.cpp
EvaluacionVeterinaria	EvaluacionVeterinaria.h / EvaluacionVeterinaria.cpp

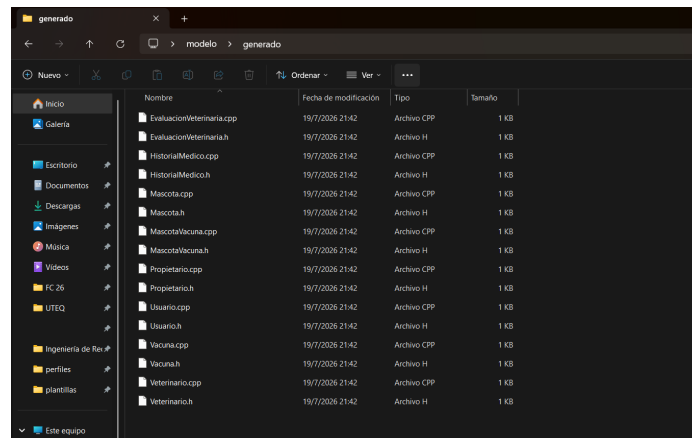


Figura 6: Árbol de archivos generados en la carpeta generado/ a partir del modelo UML en BOUML.

8. Verificación del código generado

8.1. Compilación y ejecución

El proyecto generado por BOUML se compiló utilizando el compilador **g++** (MinGW) mediante el siguiente comando, ejecutado sobre la carpeta **generado/**:

```
g++ Usuario.cpp Propietario.cpp Veterinario.cpp Mascota.cpp
HistorialMedico.cpp Vacuna.cpp MascotaVacuna.cpp
EvaluacionVeterinaria.cpp main.cpp -o mundipets_demo
```

Como unidad mínima demostrable, se elaboró un archivo `main.cpp` que instancia un objeto *Propietario* y un objeto *Mascota*, enlaza la asociación entre ambos mediante los punteros generados por BOUML (`mascota1.propietario` y `propietario1.mascotas`), y verifica en tiempo de ejecución la navegación bidireccional de dicha asociación. La compilación finalizó sin errores, generando el ejecutable `mundipets_demo.exe`, cuya salida en consola confirma que la relación entre ambas entidades quedó correctamente enlazada en memoria, tal como se muestra en la Figura 7.

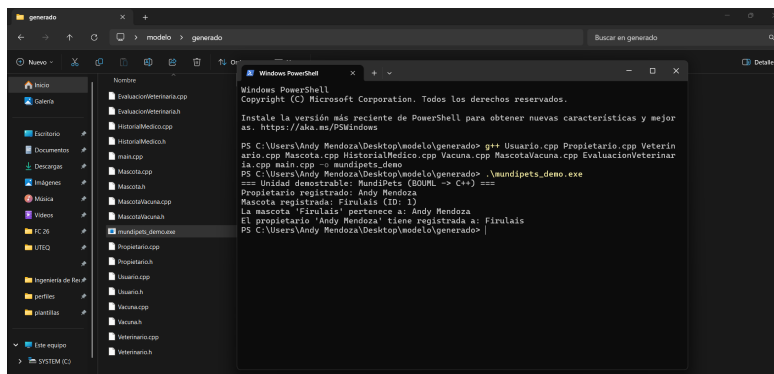


Figura 7: Salida en consola del ejecutable generado, confirmando la instanciación y asociación correcta de Propietario y Mascota.

8.2. Correspondencia modelo → código

La Tabla 5 documenta la trazabilidad entre los elementos definidos en el diagrama de clases de origen (Sección 4) y los artefactos concretos que el generador C++ de BOUML produjo en la carpeta generado/.

Tabla 5: Correspondencia entre elementos del modelo UML y artefactos C++ generados.

Elemento UML	Artefacto C++ generado
Clase Mascota	Mascota.h / Mascota.cpp
Atributo idMascota (long, público)	Campo público long idMascota; en Mascota.h
Atributo nombre (string, público)	Campo público string nombre; en Mascota.h
Generalización Usuario → Propietario	class Propietario : public Usuario en Propietario.h
Asociación 1–N Propietario (1) ↔ Mascota (0..*)	Puntero Propietario * propietario; en Mascota.h, y Mascota * mascotas; en Propietario.h
Composición Mascota ♦ HistorialMedico (1:1)	Miembro por valor HistorialMedico historial; en Mascota.h
Asociación 1–N Mascota (1) ↔ EvaluacionVeterinaria (0..*)	Puntero EvaluacionVeterinaria * evaluaciones; en Mascota.h
Asociación 1–N Veterinario (1) ↔ EvaluacionVeterinaria (0..*)	Puntero EvaluacionVeterinaria * evaluacionesRegistradas; en Veterinario.h
Clase asociativa MascotaVacuna	MascotaVacuna.h / MascotaVacuna.cpp; a diferencia de lo planificado, el generador no produjo ninguna referencia por puntero ni por contenedor <code>vector</code> entre <code>Vacuna.h</code> y <code>MascotaVacuna</code> : el vínculo quedó resuelto únicamente mediante los atributos <code>idMascota</code> e <code>idVacuna</code> (tipo <code>long</code>) declarados dentro de <code>MascotaVacuna.h</code> , a modo de clave foránea, sin navegabilidad por objeto (ver limitación en Sección 7.3)

8.3. Limitaciones detectadas

Durante el proceso de generación y verificación se identificaron seis limitaciones concretas del generador C++ de BOUML frente al alcance planificado en la Sección 6.2:

- **Plantilla de asociación dependiente de un estereotipo vacío.** La plantilla por defecto incluía el marcador de estereotipo entre símbolos de plantilla de tipo. Cuando la asociación no tenía un estereotipo asignado, se sustituía por una cadena vacía dejando los símbolos de apertura y cierre sueltos en el código generado, provocando errores de compilación.
- **Asociaciones sin nombre de rol generadas como atributos anónimos.** Algunas asociaciones quedaron registradas sin un nombre de rol asignado, lo que BOUML tradujo como una declaración de atributo sin identificador, inválida en C++. Estas asociaciones debieron eliminarse individualmente.
- **Contenedores `vector` no aplicados al entregable final.** Aunque la pestaña C++[5] del generador permite resolver tipos contenedores externos, el equipo optó por conservar la plantilla nativa de puntero simple en todas las asociaciones con multiplicidad múltiple. En consecuencia, el código final (`Propietario.h`, `Mascota.h`, `Veterinario.h`) no contiene ningún `vector`, y la navegación 1–N declarada en el modelo queda representada de forma incompleta: un puntero simple solo puede apuntar a un elemento a la vez.
- **Clase asociativa `MascotaVacuna` sin navegabilidad por objeto.** A diferencia de lo previsto, el generador no produjo ningún puntero entre `Vacuna` y `MascotaVacuna` (ni viceversa); el vínculo quedó resuelto únicamente mediante los atributos `idMascota` e `idVacuna` (tipo `long`) dentro de `MascotaVacuna.h`, a modo de clave foránea sin relación de objetos en memoria.
- **Ausencia de métodos de acceso (getters y setters) generados por defecto.** BOUML no genera automáticamente estos métodos para los atributos; requiere definirlos explícitamente. Se optó por exponer como públicos los atributos estrictamente necesarios para la demostración como una simplificación deliberada.
- **Cuerpo de operación generado sin sentencia `return`.** La operación `float Mascota::calcularDosisRecomendada()` incorporada durante el roundtrip (Sección 9), se generó con el cuerpo vacío. El compilador g++ advierte “*no return statement in function returning non-void*”; el código compila y enlaza, pero el valor devuelto por esta operación es indefinido hasta que el equipo complete manualmente su lógica de negocio.

9. Cierre del ciclo (roundtrip)

9.1. Cambio aplicado al modelo

Para demostrar operativamente la bidireccionalidad y el cierre del ciclo de desarrollo dirigido por modelos (MDD), se planificó y ejecutó una modificación arquitectónica controlada directamente sobre el diseño estructural del subsistema sanitario [8]. El cambio consistió en dotar a la entidad central *Mascota* de la capacidad de almacenar características físicas e incorporar comportamiento lógico para el negocio clínico [7].

Específicamente, dentro del lienzo de BOUML se incorporó el atributo privado `pesoKg` con tipo primitivo `float`, y se añadió la operación pública `calcularDosisRecomendada()` con tipo de retorno `float` [20]. La estructura original de la entidad previa a esta evolución arquitectónica puede observarse detalladamente en el modelo global de la Figura 1 [8]. Este escenario simula una evolución real de requisitos en el PFC *MundiPets*, permitiendo evaluar la respuesta del motor ante la alteración del metamodelo de origen sin necesidad de reescribir los archivos desde cero [15, 17].

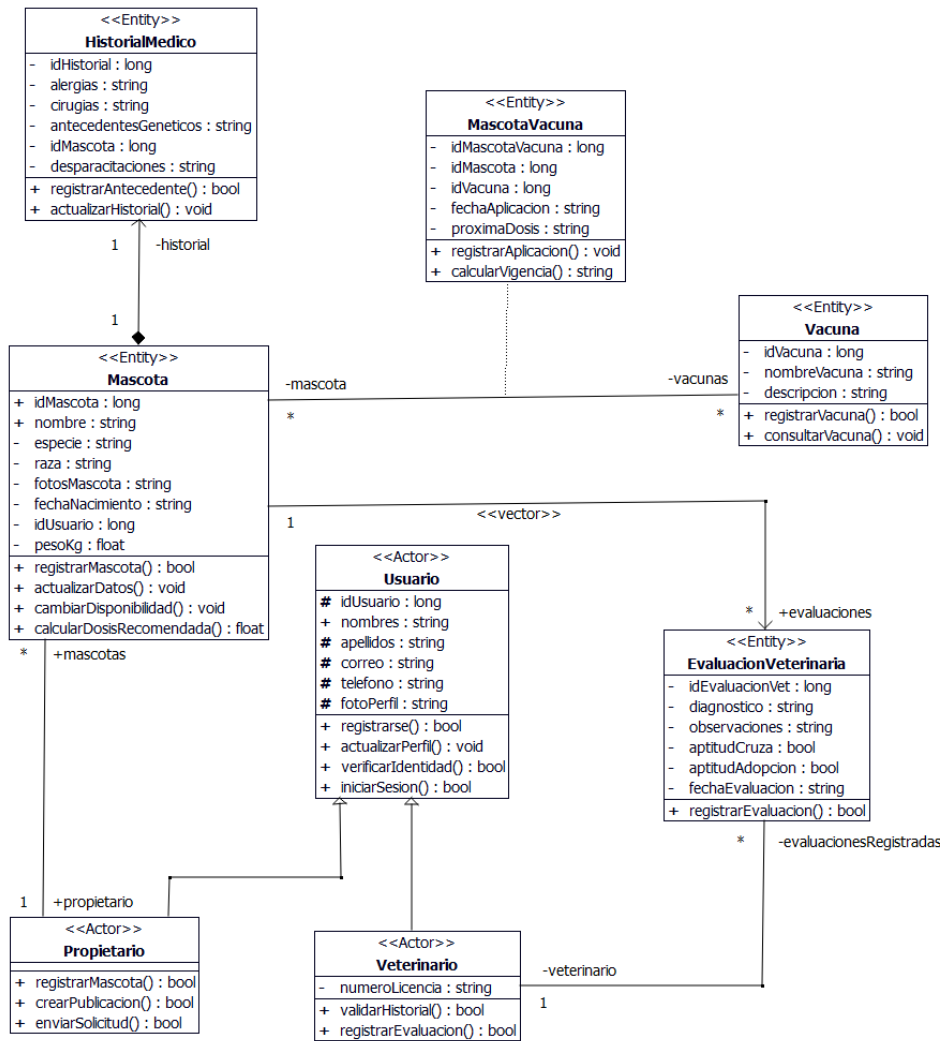


Figura 8: Diagrama de clases de la entidad Mascota en BOUML tras incorporar el atributo pesoKg y la operación calcularDosisRecomendada.

9.2. Evidencia antes/después

Una vez aplicado el cambio en el modelo gráfico, la sincronización bidireccional se ejecutó activando el comando específico *Roundtrip engineering C++* desde el menú contextual del paquete de clases en BOUML [20]. A diferencia de una generación unidireccional simple, este motor lee primero el estado del código físico en el disco para mapear los bloques manuales existentes y, posteriormente, inyecta las nuevas firmas lógicas del Browser de manera consistente [8, 20]. Al ingresar al archivo de cabecera *Mascota.h*, se constató la inserción exitosa de las firmas correspondientes: la variable encapsulada *float pesoKg*; en la zona privada de la clase y la declaración del método *float calcularDosisRecomendada()*; en la sección pública [20].

Un aspecto crítico verificado de forma empírica en este proceso fue el comportamiento del generador frente a la política de **preservación de bloques protegidos** [20]. El código manual previamente embebido por el equipo de desarrollo para enlazar las asociaciones de punteros dinámicos no sufrió ninguna alteración ni borrado accidental [25]. El analizador de BOUML aisló las modificaciones estructurales y respetó estrictamente las líneas de código humano situadas entre los marcadores de preservación del entorno, demostrando que el flujo *round-trip* ejecutado es consistente, reproducible y seguro para el desarrollo incremental del backend en C++ [8, 20].

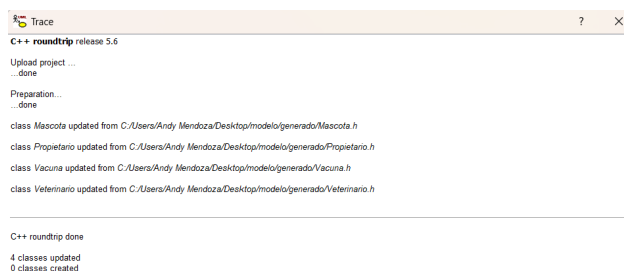


Figura 9: Evidencia de la traza de comparación (diff) reflejando la inserción exacta del nuevo atributo y método en Mascota.h respetando los bloques protegidos.

10. Reparto del trabajo

Para certificar el cumplimiento pleno de las directrices de trabajo equitativo y asegurar la transparencia en la contribución individual, la distribución de responsabilidades lógicas del proyecto se gobernó bajo un esquema de asignación de fases atómicas [8]. Cada integrante del equipo ejecutó sus aportaciones empleando de forma estricta su identidad digital real y cuenta institucional en la plataforma de control de versiones GitHub, vinculando de manera verificable los registros del historial con el desarrollo físico de este reporte y de los componentes de código fuente generados en C++ [8].

La Tabla 6 expone formalmente la distribución de liderazgo técnico y autorías por cada sección consolidada en el informe final.

Tabla 6: Distribución formal de responsabilidades lógicas por integrante.

Capítulo	Sección / Fase del Proyecto	Integrante Responsable
1	Marco teórico (Conceptos base y estándares)	Sánchez C. Gary / Chavarría C. Mel
2	Justificación de la selección de BOUML	Chavarría Cuenca Tahiny Mel
3	Descripción del subsistema del PFC modelado	Chavarría Cuenca Tahiny Mel
4	Modelo UML de origen (Lienzo y validación)	Mendoza Párraga Andy Johel
5	Configuración del generador M2T C++	Sánchez Cornejo Gary Alberto
6	Proceso de generación automatizada	Sánchez Cornejo Gary Alberto
7	Verificación del código generado y pruebas g++	Sánchez Cornejo Gary Alberto
8	Cierre del ciclo (<i>Roundtrip engineering C++</i>)	Mendoza Párraga Andy Johel
9	Reparto del trabajo y consistencia Git	Mendoza Párraga Andy Johel
10	Conclusiones funcionales finales	Equipo H

10.1. Trazabilidad de contribuciones técnicas e historial en GitHub

En concordancia con las restricciones de auditoría académica, las contribuciones técnicas se desarrollaron de manera incremental a través de múltiples confirmaciones de cambio en el servidor. A continuación, se detallan las tareas específicas lideradas por cada estudiante, junto con la dirección URL configurada para auditar el listado completo y cronológico de sus respectivos *commits*:

■ Mendoza Párraga Andy Johel (Titular del PFC)

- *Foco de contribución técnica:* Diseño, tipificación y normalización del diagrama de clases estructural y casos de uso en el archivo nativo del proyecto; ejecución de controles sintácticos mediante la auditoría *Uml projection*; planeación e inyección interactiva del cambio controlado de requisitos en el modelo (atributo `pesoKg` y método de dosis) ejecutando el motor de *Roundtrip engineering C++* para verificar el resguardo de bloques protegidos.
- *Historial completo de Commits:*
<https://github.com/andymendozap03/PFC-MundiPets-MDD-EquipoH-BOUML/commits?author=andymendozap03>

■ Sánchez Cornejo Gary Alberto

- *Foco de contribución técnica:* Fundamentación teórica de las transformaciones M2M/M2T, metamodelos y plantillas del marco conceptual; parametrización global del motor de generación física (rutas de salida, extensiones `.h.cpp`, guardas de inclusión) y evaluación de la representación de multiplicidades múltiples mediante punteros simples frente a contenedores `std::vector` (documentada como limitación en la Sección 7.3); codificación de la unidad demostrable `main.cpp` y pruebas de compilación nativa con `g++`.
- *Historial completo de Commits:*
<https://github.com/andymendozap03/PFC-MundiPets-MDD-EquipoH-BOUML/commits?author=gsanchezc6-beep>

■ Chavarría Cuenca Tahiny Mel

- *Foco de contribución técnica:* Redacción y fundamentación del marco justificativo de la herramienta frente a las restricciones del dominio clínico y la norma de requisitos ISO/IEC/IEEE 29148:2018; diseño de la matriz

comparativa de viabilidad contra la alternativa descartada Modelo 5.4.1 y redacción de la sección contextual del ecosistema web de la plataforma.

- *Historial completo de Commits:*

<https://github.com/andymendozap03/PFC-MundiPets-MDD-EquipoH-BOUML/commits?author=TahinyMel>

10.2. Log de aportaciones individuales (Insights Contributors)

Como evidencia objetiva de la participación continua, balanceada y distribuida a lo largo de todo el ciclo de vida del desarrollo, la Figura 10 expone la métrica agregada de aportes extraída directamente de la sección de analíticas del repositorio en la nube.

Se destaca una correlación exacta entre los roles declarados y las métricas del servidor: la estudiante Chavarría lidera la frecuencia transaccional con 16 confirmaciones, el estudiante Mendoza acumula el mayor volumen de adiciones estructurales con 2,190 líneas debido a la inyección del metamodelo y los esqueletos físicos de C++, y el estudiante Sánchez consolida los componentes técnicos complementarios mediante 6 confirmaciones en la etapa final de pruebas.

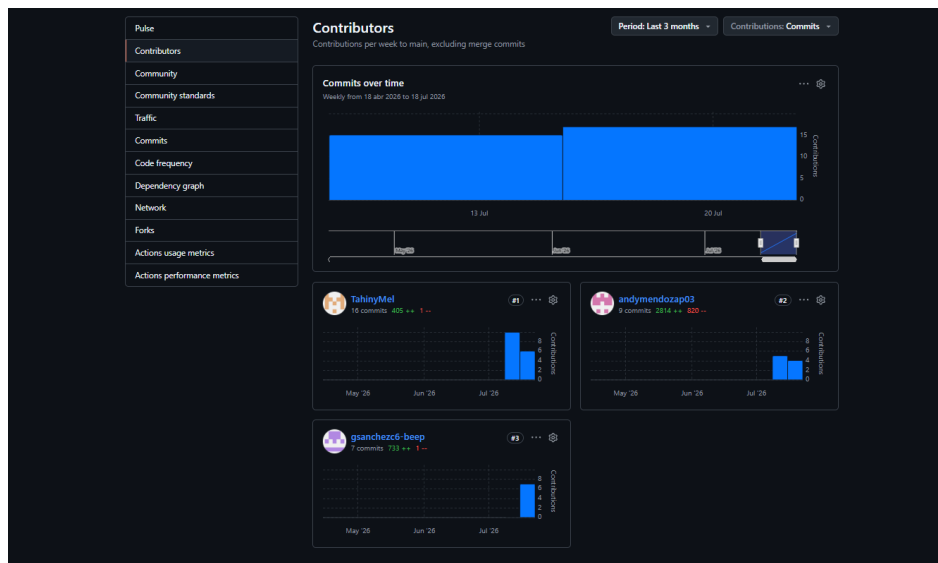


Figura 10: Métricas de aportes y commits individuales extraídas de la sección Insights/Contributors de GitHub.

11. Conclusiones

Luego de culminar con todo el ciclo práctico de este proyecto, como equipo pudimos constatar el valor real que tiene el desarrollo guiado por modelos para nuestro sistema MundiPets. La principal ventaja de aplicar este enfoque fue la reducción drástica del trabajo repetitivo en el backend, ya que pudimos automatizar la creación de las ocho clases sanitarias con sus tipos de datos correspondientes. Al dejar que la herramienta se encargue de generar la estructura base, no solo ahorramos tiempo valioso, sino que eliminamos la posibilidad de arrastrar errores humanos al escribir las declaraciones y las herencias nativas en C++. El diseño dejó de ser un simple dibujo decorativo para convertirse en el motor directo de la programación.

El aprendizaje más importante que nos llevamos de esta experiencia es entender cómo funciona el ciclo completo de sincronización bidireccional, conocido como roundtrip engineering. Al principio pensábamos que cualquier cambio en el lienzo destruiría los avances manuales del backend, pero comprobar el funcionamiento de los bloques protegidos de BOUML nos dio una perspectiva completamente diferente. Ver cómo el generador es capaz de inyectar nuevos atributos como el peso de la mascota sin tocar una sola línea de la lógica que programamos a mano nos enseñó que se puede mantener un diseño limpio, ordenado y perfectamente sincronizado a lo largo de todo el ciclo de desarrollo.

Por último, es necesario reconocer que experimentamos limitaciones técnicas importantes durante las pruebas. BOUML es excelente para armar el esqueleto orientado a objetos, pero nos obligó a realizar ajustes manuales debido a que no genera de forma automática los métodos de acceso para los atributos, y sus plantillas por defecto a veces crean código incompleto si las asociaciones no tienen roles definidos. Además, el alcance de la herramienta se limita estrictamente al código lógico de C++, lo que significa que tanto la base de datos como las pantallas de la interfaz gráfica quedan completamente fuera de su control y dependen por entero de nuestro trabajo manual en las siguientes fases del proyecto.

12. Referencias

- [1] M. Brambilla, J. Cabot y M. Wimmer, *Model-Driven Software Engineering in Practice* (Synthesis Lectures on Software Engineering), 2nd. Morgan & Claypool Publishers, 2017. DOI: 10.2200/S00751ED2V01Y201701SWE001
- [2] D. C. Schmidt, “Guest Editor’s Introduction: Model-Driven Engineering,” *Computer*, vol. 39, n.º 2, págs. 25-31, 2006. DOI: 10.1109/MC.2006.58
- [3] B. Selic, “The pragmatics of model-driven development,” *IEEE Software*, vol. 20, n.º 5, págs. 19-25, 2003. DOI: 10.1109/MS.2003.1231146
- [4] J. Whittle, J. Hutchinson y M. Rouncefield, “The state of practice in model-driven engineering,” *IEEE Software*, vol. 31, n.º 3, págs. 79-85, 2014. DOI: 10.1109/MS.2013.65
- [5] D. D. Ruscio, D. S. Kolovos, J. de Lara, A. Pierantonio, M. Tisi y M. Wimmer, “Low-Code Development and Model-Driven Engineering: Two Sides of the Same Coin?” *Software and Systems Modeling*, vol. 21, n.º 2, págs. 437-446, 2022. DOI: 10.1007/s10270-021-00970-2 dirección: <https://doi.org/10.1007/s10270-021-00970-2>
- [6] A. Bucaioni, A. Cicchetti y F. Ciccozzi, “Modelling in Low-Code Development: A Multi-Vocal Systematic Review,” *Software and Systems Modeling*, vol. 21, n.º 5, págs. 1959-1981, 2022. DOI: 10.1007/s10270-021-00964-0 dirección: <https://doi.org/10.1007/s10270-021-00964-0>
- [7] Equipo MundiPets, “Proyecto Fin de Curso: Entrega 1B—Documento ERS/SRS Parcial,” Universidad Técnica Estatal de Quevedo, Quevedo, Ecuador, Especificación de Requisitos de Software, 2026, Versión 2.0.
- [8] G. G. Ulloa, “Guía y Rúbrica de Evaluación: Herramientas que Generan Código Fuente a Partir de Modelos UML,” Universidad Técnica Estatal de Quevedo, Quevedo, Ecuador, Guía académica, 2026.
- [9] Object Management Group, “Model Driven Architecture (MDA) Guide, Revision 2.0,” Object Management Group, inf. téc. ormsc/2014-06-01, jun. de 2014. dirección: <https://www.omg.org/cgi-bin/doc?ormsc/2014-06-01>
- [10] N. Silega et al., “Transformation From CIM to PIM: A Systematic Mapping,” *IEEE Access*, vol. 10, 2022. DOI: 10.1109/ACCESS.2022.3201556
- [11] L. Huning y E. Pulvermüller, “Automatic Code Generation of Safety Mechanisms in Model-Driven Development,” *Electronics*, vol. 10, n.º 24, pág. 3150, 2021. DOI: 10.3390/electronics10243150 dirección: <https://doi.org/10.3390/electronics10243150>
- [12] A. D. Durai, M. Ganesh, R. M. Mathew y D. K. Anguraj, “A Novel Approach with an Extensive Case Study and Experiment for Automatic Code Generation from the XMI Schema of UML Models,” *The Journal of Supercomputing*, vol. 78, n.º 6, págs. 7677-7699, 2022. DOI: 10.1007/s11227-021-04164-x dirección: <https://doi.org/10.1007/s11227-021-04164-x>
- [13] Object Management Group, “Meta Object Facility (MOF) 2.0 Query/View/Transformation Specification, Version 1.3,” Object Management Group, inf. téc. formal/16-06-03, 2016.
- [14] S. Höppner, Y. Haas, M. Tichy y K. Juhnke, “Advantages and disadvantages of (dedicated) model transformation languages: A qualitative interview study,” *Empirical Software Engineering*, vol. 27, n.º 6, nov. de 2022. DOI: 10.1007/s10664-022-10194-7
- [15] Object Management Group, *MOF Model to Text Transformation Language, Version 1.0*, Object Management Group, ene. de 2008. dirección: <https://www.omg.org/spec/MOFM2T/1.0>
- [16] L. Jiménez-Navajas, R. Pérez-Castillo y M. Piattini, “Code Generation for Classical-Quantum Software Systems Modeled in UML,” *Software and Systems Modeling*, vol. 24, n.º 3, págs. 795-821, 2025. DOI: 10.1007/s10270-024-01259-w dirección: <https://doi.org/10.1007/s10270-024-01259-w>
- [17] Object Management Group, *OMG Unified Modeling Language (OMG UML), Version 2.5.1*, Object Management Group, dic. de 2017. dirección: <https://www.omg.org/spec/UML/2.5.1>
- [18] P. Di Felice, G. Paolone, R. Paesani y M. Marinelli, “Design and Implementation of a Metadata Repository about UML Class Diagrams. A Software Tool Supporting the Automatic Feeding of the Repository,” *Electronics*, vol. 11, n.º 2, pág. 201, 2022. DOI: 10.3390/electronics11020201
- [19] J. Di Rocco, D. Di Ruscio, C. Di Sipio, P. T. Nguyen y A. Pierantonio, “MemoRec: A Recommender System for Assisting Modelers in Specifying Metamodels,” *Software and Systems Modeling*, vol. 22, n.º 1, págs. 203-223, 2022. DOI: 10.1007/s10270-022-00994-2
- [20] B. Pagès, *BOUML Features*, Documentación oficial de BOUML, Consultado en julio de 2026, 2026. dirección: <https://www.bouml.fr/features.html>
- [21] A. Kraas, “On the Automation-Supported Derivation of Domain-Specific UML Profiles Considering Static Semantics,” *Software and Systems Modeling*, vol. 21, n.º 1, págs. 51-79, 2022. DOI: 10.1007/s10270-021-00890-1
- [22] T. Górski, “UML Profile for Messaging Patterns in Service-Oriented Architecture, Microservices, and Internet of Things,” *Applied Sciences*, vol. 12, n.º 24, pág. 12 790, 2022. DOI: 10.3390/app122412790

- [23] G. Paolone, R. Paesani, M. Marinelli y P. D. Felice, “Empirical Assessment of the Quality of MVC Web Applications Returned by xGenerator,” *Computers*, vol. 10, 2021. DOI: 10.3390/computers dirección: <https://doi.org/10.3390/computers>
- [24] U. Kannengiesser, J. Frysak, C. Sary, F. Krenn et al., “Developing an Engineering Tool for Cyber-Physical Production Systems,” *e & i Elektrotechnik und Informationstechnik*, vol. 138, págs. 330-340, 2021. DOI: 10.1007/s00502-021-00911-3 dirección: <https://doi.org/10.1007/s00502-021-00911-3>
- [25] R. Jongeling, F. Ciccozzi, J. Carlson, A. Cicchetti et al., “Consistency Management in Industrial Continuous Model-Based Development Settings: A Reality Check,” *Software and Systems Modeling*, vol. 21, págs. 1511-1530, 2022. DOI: 10.1007/s10270-022-01000-5 dirección: <https://doi.org/10.1007/s10270-022-01000-5>
- [26] M. Ozkaya y F. Erata, “A Survey on the Practical Use of UML for Different Software Architecture Viewpoints,” *Information and Software Technology*, vol. 121, pág. 106275, 2020. DOI: 10.1016/j.infsof.2020.106275
- [27] M. Alharbi y M. Alshayeb, “Automatic Code Generation Techniques: A Systematic Literature Review,” *Automated Software Engineering*, vol. 33, n.º 1, 2026, Art. no. 4. DOI: 10.1007/s10515-025-00551-3 dirección: <https://doi.org/10.1007/s10515-025-00551-3>
- [28] D. Lehner et al., “Model-Driven Engineering for Digital Twins: A Systematic Mapping Study,” *Software and Systems Modeling*, vol. 24, págs. 1339-1377, 2025. DOI: 10.1007/s10270-025-01264-7 dirección: <https://doi.org/10.1007/s10270-025-01264-7>
- [29] N. Kahani, M. Bagherzadeh, J. R. Cordy, J. Dingel y D. Varró, “Survey and classification of model transformation tools,” *Software and Systems Modeling*, vol. 18, n.º 4, págs. 2361-2397, 2019. DOI: 10.1007/s10270-018-0665-6 dirección: <https://doi.org/10.1007/s10270-018-0665-6>
- [30] B. Pagès, *BOUML: A Free UML Tool Box*, Sitio web oficial de BOUML, Consultado en julio de 2026, 2025. dirección: <https://www.bouml.fr/>
- [31] Modelio Open Source, *Modelio: The Open Source Modeling Environment*, Sitio web oficial de Modelio, Versión 5.4.1. Consultado en julio de 2026, 2026. dirección: <https://www.modelio.org/>

Anexos

Anexo A — Especificación detallada de Requisitos Funcionales del sistema

A continuación, se presenta la especificación atómica detallada de la totalidad de los requisitos funcionales del sistema MundiPets, estructurados bajo los ocho atributos de gestión establecidos por el estándar ISO/IEC/IEEE 29148:2018.

Requisitos Funcionales del Subsistema de Gestión Sanitaria

RF-01 – Registrar mascota

Atributo	Valor
Identificador	RF-01
Nombre	Registrar mascota
Descripción	El sistema deberá permitir al propietario registrar una mascota ingresando su información general, incluyendo nombre, especie, raza, sexo, edad, estado reproductivo y fotografías para su identificación dentro de la plataforma.
Prioridad (MoSCoW)	Must
Fuente / Evidencia	EV-02, EV-03, EV-05, EV-06, EV-07, EV-08
Estabilidad	Alta
Estado	Aprobado
Criterio de Verificación	Registrar una mascota con datos válidos y comprobar que el perfil queda almacenado y puede consultarse posteriormente.

RF-02 – Gestionar historial médico de la mascota

Atributo	Valor
Identificador	RF-02
Nombre	Gestionar historial médico de la mascota
Descripción	El sistema deberá permitir registrar y actualizar el historial médico de una mascota, incluyendo vacunas, desparasitaciones, enfermedades, alergias, cirugías, tratamientos y controles veterinarios.
Prioridad (MoSCoW)	Must
Fuente / Evidencia	EV-01, EV-02, EV-03, EV-04, EV-05, EV-06, EV-07
Estabilidad	Alta
Estado	Aprobado
Criterio de Verificación	Registrar nuevos antecedentes médicos y verificar que aparecen correctamente dentro del historial clínico de la mascota.

RF-03 – Consultar perfil completo de una mascota

Atributo	Valor
Identificador	RF-03
Nombre	Consultar perfil completo de una mascota
Descripción	El sistema deberá permitir visualizar el perfil completo de una mascota mostrando su información general, historial médico autorizado, fotografías, comportamiento, vacunas, edad, raza para procesos de adopción o cruce.
Prioridad (MoSCoW)	Must
Fuente / Evidencia	EV-02, EV-04, EV-05, EV-06, EV-07, EV-08
Estabilidad	Alta
Estado	Aprobado
Criterio de Verificación	Consultar una mascota registrada y comprobar que se muestra toda la información autorizada.

RF-09 – Validar la información médica de una mascota

Atributo	Valor
Identificador	RF-09
Nombre	Validar la información médica de una mascota
Descripción	El sistema deberá permitir que una veterinaria autorizada valide la información médica registrada de una mascota, indicando que los datos han sido verificados por un profesional.
Prioridad (MoSCoW)	Must
Fuente / Evidencia	EV-03, EV-05, EV-07
Estabilidad	Alta
Estado	Aprobado
Criterio de Verificación	Validar el historial médico de una mascota y comprobar que el sistema muestre el estado de información validada.

RF-14 – Gestionar carnet de vacunación

Atributo	Valor
Identificador	RF-14
Nombre	Gestionar carnet de vacunación
Descripción	El sistema deberá permitir registrar, actualizar y consultar el carnet de vacunación de cada mascota, mostrando el historial de vacunas aplicadas, fechas, próximas dosis y su estado de vigencia.
Prioridad (MoSCoW)	Must
Fuente / Evidencia	EV-02, EV-03, EV-05, EV-06, EV-07
Estabilidad	Alta
Estado	Aprobado
Criterio de Verificación	Registrar una vacuna y comprobar que aparezca correctamente en el carnet de vacunación.

RF-16 – Gestionar antecedentes genéticos y parentesco de la mascota

Atributo	Valor
Identificador	RF-16
Nombre	Gestionar antecedentes genéticos y parentesco de la mascota
Descripción	El sistema deberá permitir registrar y consultar los antecedentes genéticos y el grado de parentesco de una mascota, incluyendo información sobre enfermedades hereditarias y linaje para apoyar los procesos de cruce responsable y la evaluación de compatibilidad.
Prioridad (MoSCoW)	Must
Fuente / Evidencia	EV-01, EV-03, EV-05, EV-07
Estabilidad	Alta
Estado	Aprobado
Criterio de Verificación	Registrar antecedentes genéticos para una mascota y comprobar que la información quede almacenada.

Requisitos Funcionales Transaccionales y de Soporte

RF-04 – Buscar y filtrar mascotas

Atributo	Valor
Identificador	RF-04
Nombre	Buscar y filtrar mascotas
Descripción	El sistema deberá permitir buscar mascotas aplicando filtros como especie, raza, edad, ubicación, estado de salud y disponibilidad para adopción o cruce.
Prioridad (MoSCoW)	Should
Fuente / Evidencia	EV-07, EV-08
Estabilidad	Media
Estado	Aprobado
Criterio de Verificación	Aplicar diferentes filtros y comprobar que los resultados corresponden con los criterios seleccionados.

RF-05 – Gestionar solicitudes de adopción

Atributo	Valor
Identificador	RF-05
Nombre	Gestionar solicitudes de adopción
Descripción	El sistema deberá permitir que un usuario interesado envíe solicitudes de adopción y que el propietario pueda aceptarlas o rechazarlas después de revisar la información del solicitante.
Prioridad (MoSCoW)	Must
Fuente / Evidencia	EV-01, EV-02, EV-03, EV-05, EV-06, EV-08
Estabilidad	Alta
Estado	Aprobado
Criterio de Verificación	Registrar una solicitud y comprobar que el propietario puede gestionarla correctamente.

RF-06 – Evaluar compatibilidad entre mascotas para cruce

Atributo	Valor
Identificador	RF-06
Nombre	Evaluar compatibilidad entre mascotas para cruce
Descripción	El sistema deberá evaluar la compatibilidad entre dos mascotas registradas para procesos de cruce considerando la raza, edad, estado de salud, antecedentes genéticos, parentesco, comportamiento e historial médico, mostrando al usuario el resultado obtenido para apoyar la toma de decisiones.
Prioridad (MoSCoW)	Must
Fuente / Evidencia	EV-01, EV-02, EV-03, EV-04, EV-05, EV-06, EV-07, EV-08
Estabilidad	Alta
Estado	Aprobado
Criterio de Verificación	Seleccionar dos mascotas y comprobar que el sistema presenta la información de compatibilidad.

RF-07 – Sistema de mensajería entre usuarios

Atributo	Valor
Identificador	RF-07
Nombre	Sistema de mensajería entre usuarios
Descripción	El sistema deberá habilitar un canal de comunicación entre dos usuarios registrados cuando exista un proceso de adopción o cruce activo entre ellos.
Prioridad (MoSCoW)	Could
Fuente / Evidencia	EV-02, EV-07, EV-08
Estabilidad	Media
Estado	Aprobado
Criterio de Verificación	Enviar un mensaje entre dos usuarios autorizados y verificar que ambos puedan visualizarlo.

RF-08 – Gestionar solicitudes de cruce

Atributo	Valor
Identificador	RF-08
Nombre	Gestionar solicitudes de cruce
Descripción	El sistema deberá permitir que los propietarios soliciten procesos de cruce entre mascotas y que el propietario de la otra mascota pueda aceptar o rechazar la solicitud con base en la información disponible.
Prioridad (MoSCoW)	Must
Fuente / Evidencia	EV-01, EV-02, EV-03, EV-05, EV-06, EV-07
Estabilidad	Alta
Estado	Aprobado
Criterio de Verificación	Registrar una solicitud de cruce y verificar que el propietario receptor pueda gestionarla.

RF-10 – Gestionar recordatorios de controles preventivos

Atributo	Valor
Identificador	RF-10
Nombre	Gestionar recordatorios de controles preventivos
Descripción	El sistema deberá enviar un recordatorio al propietario de la mascota antes de la fecha programada de una cita veterinaria y registrar el envío de la notificación.
Prioridad (MoSCoW)	Should
Fuente / Evidencia	EV-02, EV-04, EV-06
Estabilidad	Media
Estado	Aprobado
Criterio de Verificación	Configurar una alerta de control próximo y verificar el registro de envío de la notificación.

RF-11 – Administrar la privacidad de la información

Atributo	Valor
Identificador	RF-11
Nombre	Administrar la privacidad de la información
Descripción	El sistema deberá permitir que el propietario determine qué información de su mascota y de su perfil podrá ser visualizada por otros usuarios, protegiendo los datos considerados sensibles.
Prioridad (MoSCoW)	Must
Fuente / Evidencia	EV-02, EV-05, EV-06, EV-07, EV-08
Estabilidad	Alta
Estado	Aprobado
Criterio de Verificación	Modificar la configuración de privacidad y comprobar que los datos restringidos dejen de ser visibles.

RF-12 – Verificar la identidad de los usuarios

Atributo	Valor
Identificador	RF-12
Nombre	Verificar la identidad de los usuarios
Descripción	El sistema deberá validar la información obligatoria proporcionada por el usuario antes de aprobar el proceso de verificación de identidad.
Prioridad (MoSCoW)	Must
Fuente / Evidencia	EV-01, EV-03, EV-07, EV-08
Estabilidad	Alta
Estado	Aprobado
Criterio de Verificación	Registrar un usuario con la documentación requerida y comprobar que el sistema cambie el estado de verificación.

RF-13 – Registrar publicaciones de mascotas

Atributo	Valor
Identificador	RF-13
Nombre	Registrar publicaciones de mascotas
Descripción	El sistema deberá permitir al propietario publicar una mascota indicando si estará disponible para adopción, cruza o ambas opciones, incluyendo toda la información autorizada del perfil de la mascota.
Prioridad (MoSCoW)	Must
Fuente / Evidencia	EV-01, EV-02, EV-04, EV-06, EV-07, EV-08
Estabilidad	Alta
Estado	Aprobado
Criterio de Verificación	Publicar una mascota y comprobar que aparezca correctamente en el listado seleccionado.

RF-15 – Consultar el historial de procesos de adopción y cruza

Atributo	Valor
Identificador	RF-15
Nombre	Consultar el historial de procesos de adopción y cruza
Descripción	El sistema deberá permitir al propietario consultar el historial de solicitudes y procesos de adopción y cruza asociados a sus mascotas, incluyendo su estado y fecha de realización.
Prioridad (MoSCoW)	Could
Fuente / Evidencia	EV-01, EV-02, EV-04, EV-06
Estabilidad	Media
Estado	Aprobado
Criterio de Verificación	Acceder al panel histórico del propietario y verificar las trazas y estados de solicitudes pasadas.

RF-17 – Validar imágenes de publicaciones

Atributo	Valor
Identificador	RF-17
Nombre	Validar imágenes de publicaciones
Descripción	El sistema deberá validar las imágenes cargadas por los usuarios verificando que cumplan con las políticas de contenido establecidas por la plataforma antes de permitir su publicación.
Prioridad (MoSCoW)	Should
Fuente / Evidencia	EV-01, EV-03, EV-05, EV-07, EV-08
Estabilidad	Media
Estado	Aprobado
Criterio de Verificación	Cargar archivos multimedia conformes y no conformes, y comprobar la respuesta del módulo de validación.

Anexo B — Estructura del repositorio GitHub

A continuación, se detalla la organización física y jerárquica de la raíz del repositorio oficial en GitHub:

```
PFC-MundiPets-MDD-EquipoH-BOUML/
|   README.md
|
|-- aportes_individuales/           (Aportes conceptuales y teóricos)
|   |-- CHAVARRIA CUENCA TAHINY MEL
|       Aporte_MelChavarria.pdf
|       referencias.bib
|
|   |-- MENDOZA PARRAGA ANDY JOHEL
|       AndyMendoza_Aporte.pdf
|       referencias.bib
|
|   \-- SANCHEZ CORNEJO GARY ALBERTO
|       MarcoTeorico_ERS.pdf
|       Referencias_MarcoTeorico.bib
|
|-- docs/                           (Soporte físico del manuscrito)
|   |   informe.pdf
|   |   informe.tex
|   |   referencias.bib
|   |
|   \-- figuras/                   (Diagramas y capturas del informe)
|
|-- evidencias/
|   \-- capturas/                 (Capturas del proceso completo)
|
|-- exposicion/                   (Sustentación académica)
|   \-- DiapositivasBOUML.pdf
|
|-- generado/                     (16 archivos del backend de C++)
|   |   *.h / *.cpp               (8 clases generadas estructuralmente)
|   |   main.cpp                  (Unidad demostrable del sistema)
|   \-- mundipets_demo.exe        (Ejecutable de prueba g++)
|
|-- modelo/                       (Proyecto nativo del entorno CASE)
|   |   128130.diagram            (Lienzo de Casos de Uso)
|   |   134658.diagram            (Lienzo de Clases de origen)
|   |   cpp_includes
|   |   generation_settings
|   |   modelo.prj                (Árbol del metamodelo en BOUML)
|   |   stereotypes
|   |   tools
|   |
|   \-- perfiles/                 (Extensiones del metamodelo)
|       stereotypes
|
|-- plantillas/                   (Parametrización del motor M2T)
    generation_settings
```

Anexo C — Tabla de mapeo extensa (Metamodelo UML a C++)

La siguiente matriz documenta la correspondencia semántica y sintáctica detallada entre los elementos conceptuales del diagrama de clases de origen diseñado en BOUML y las estructuras de código fuente físicas resultantes en lenguaje C++ tras la ejecución del motor de ingeniería directa:

Elemento UML Origen	Estereotipo / Tipo	Estructura Física C++	Archivo de Salida
Clase Usuario	«Actor»	class Usuario {	Usuario.h
Atributo idUsuario	protected long	protected: long idUsuario;	Usuario.h
Atributo contraseña	private string	private: string contraseña;	Usuario.h
Operación iniciarSesion()	public bool	bool iniciarSesion();	Usuario.h / .cpp
Clase Propietario	«Actor»	class Propietario : public Usuario {	Propietario.h
Relación Generalización	Herencia Nativa	: public Usuario	Propietario.h
Asociación Propietario ↔ Mascota	Cardinalidad 0..*	Mascota * mascotas; (puntero simple, plantilla nativa)	Propietario.h
Clase Veterinario	«Actor»	class Veterinario : public Usuario {	Veterinario.h
Atributo numLicencia	private string	private: string numLicencia;	Veterinario.h
Clase Mascota	«Entity»	class Mascota {	Mascota.h
Atributo idMascota	public long	public: long idMascota;	Mascota.h
Atributo nombre	public string	public: string nombre;	Mascota.h
Atributo pesoKg (Mutación)	private float	private: float pesoKg;	Mascota.h
Operación calcularDosis()	public float	float calcularDosisRecomendada();	Mascota.h / .cpp
Asociación Mascota ↔ Propietario	Cardinalidad 1	Propietario* propietario;	Mascota.h
Clase HistorialMedico	«Entity»	class HistorialMedico {	HistorialMedico.h
Composición Mascota ♦ Historial	Cardinalidad 1:1 (Por Valor)	HistorialMedico historial;	Mascota.h
Clase Vacuna	«Entity»	class Vacuna {	Vacuna.h
Atributo idVacuna	private long	private: long idVacuna;	Vacuna.h
Clase Asociativa MascotaVacuna	«Entity» M:N	class MascotaVacuna {	MascotaVacuna.h
Vínculo MascotaVacuna → Mascota	No generado como referencia por objeto	long idMascota; (clave foránea por valor, sin puntero Mascota*)	MascotaVacuna.h
Vínculo MascotaVacuna → Vacuna	No generado como referencia por objeto	long idVacuna; (clave foránea por valor, sin puntero Vacuna*)	MascotaVacuna.h
Asociación Vacuna ↔ MascotaVacuna	Cardinalidad Múltiple	Sin representación en el código generado. Vacuna.h no declara ningún miembro que referencie a MascotaVacuna	Vacuna.h
Clase EvaluacionVeterinaria	«Entity»	class EvaluacionVeterinaria {	EvaluacionVeterinaria.h
Asociación Mascota → Evaluación	Unidireccional 0..*	EvaluacionVeterinaria * evaluaciones; (puntero simple, plantilla nativa)	Mascota.h
Asociación Veterinario ↔ Evaluación	Bidireccional Simple 0..*	EvaluacionVeterinaria * evaluacionesRegistradas; (puntero simple, plantilla nativa)	Veterinario.h

Anexo D — Fragmentos de código fuente representativos

A continuación se exponen las definiciones de las estructuras físicas mapeadas e instanciadas por el motor de generación M2T de BOUML para las clases del subsistema sanitario de MundiPets:

1. Clase EvaluacionVeterinaria

Código 1: Clase EvaluacionVeterinaria (EvaluacionVeterinaria.h).

```
1      #ifndef _EVALUACIONVETERINARIA_H
2      #define _EVALUACIONVETERINARIA_H
3
4      #include <string>
5      using namespace std;
6
7      class Veterinario;
8
9      // <<Entity>>
10     class EvaluacionVeterinaria {
11     private:
12         long idEvaluacionVet;
13         string diagnostico;
14         string observaciones;
15         bool aptitudCruza;
16         bool aptitudAdopcion;
17         string fechaEvaluacion;
18
19     public:
20         bool registrarEvaluacion();
21
22     private:
23         Veterinario * veterinario;
24     };
25     #endif
```

2. Clase HistorialMedico

Código 2: Clase HistorialMedico (HistorialMedico.h).

```
1      #ifndef _HISTORIALMEDICO_H
2      #define _HISTORIALMEDICO_H
3
4      #include <string>
5      using namespace std;
6
7      // <<Entity>>
8      class HistorialMedico {
9      private:
10         long idHistorial;
11         string alergias;
12         string cirugias;
13         string antecedentesGeneticos;
14         long idMascota;
15         string desparasitaciones;
16
17     public:
18         bool registrarAntecedente();
19         void actualizarHistorial();
20     };
21     #endif
```

3. Clase Mascota

Código 3: Clase Mascota (Mascota.h).

```

1      #ifndef _MASCOTA_H
2      #define _MASCOTA_H
3
4      #include <string>
5      using namespace std;
6      #include "HistorialMedico.h"
7
8      class EvaluacionVeterinaria;
9      class Propietario;
10     class Vacuna;
11
12     // <<Entity>>
13     class Mascota {
14     public:
15         long idMascota;
16         string nombre;
17
18     private:
19         string especie;
20         string raza;
21         string fotosMascota;
22         string fechaNacimiento;
23         long idUsuario;
24         HistorialMedico historial;
25
26     public:
27         EvaluacionVeterinaria * evaluaciones;
28         bool registrarMascota();
29         void actualizarDatos();
30         void cambiarDisponibilidad();
31         Propietario * propietario;
32
33     private:
34         Vacuna * vacunas;
35         float pesoKg;
36
37     public:
38         float calcularDosisRecomendada();
39     };
40     #endif

```

4. Clase MascotaVacuna

Código 4: Clase MascotaVacuna (MascotaVacuna.h).

```

1      #ifndef _MASCOTAVACUNA_H
2      #define _MASCOTAVACUNA_H
3
4      #include <string>
5      using namespace std;
6
7      // <<Entity>>
8      class MascotaVacuna {
9      private:
10         long idMascotaVacuna;
11         long idMascota;
12         long idVacuna;
13         string fechaAplicacion;
14         string proximaDosis;
15
16     public:
17         void registrarAplicacion();
18         string calcularVigencia();
19     };
20     #endif

```

5. Clase Propietario

Código 5: Clase Propietario (Propietario.h).

```
1      #ifndef _PROPIETARIO_H
2      #define _PROPIETARIO_H
3
4      #include "Usuario.h"
5
6      class Mascota;
7
8      // <<Actor>>
9      class Propietario : public Usuario {
10      public:
11          bool registrarMascota();
12          bool crearPublicacion();
13          bool enviarSolicitud();
14          Mascota * mascotas;
15      };
16      #endif
```

6. Clase Usuario

Código 6: Clase Usuario (Usuario.h).

```
1      #ifndef _USUARIO_H
2      #define _USUARIO_H
3
4      #include <string>
5      using namespace std;
6
7      // <<Actor>>
8      class Usuario {
9      protected:
10         long idUsuario;
11
12     public:
13         string nombres;
14
15     protected:
16         string apellidos;
17         string correo;
18         string telefono;
19         string fotoPerfil;
20
21     public:
22         bool registrarse();
23         void actualizarPerfil();
24         bool verificarIdentidad();
25         bool iniciarSesion();
26     };
27     #endif
```

7. Clase Vacuna

Código 7: Clase Vacuna (Vacuna.h).

```
1      #ifndef _VACUNA_H
2      #define _VACUNA_H
3
4      #include <string>
5      using namespace std;
6
7      class Mascota;
8
9      // <<Entity>>
10     class Vacuna {
11     private:
12         long idVacuna;
13         string nombreVacuna;
14         string descripcion;
15
16     public:
17         bool registrarVacuna();
18         void consultarVacuna();
19
20     private:
21         Mascota * mascota;
22     };
23     #endif
```

8. Clase Veterinario

Código 8: Clase Veterinario (Veterinario.h).

```
1      #ifndef _VETERINARIO_H
2      #define _VETERINARIO_H
3
4      #include "Usuario.h"
5      #include <string>
6      using namespace std;
7
8      class EvaluacionVeterinaria;
9
10     // <<Actor>>
11     class Veterinario : public Usuario {
12     private:
13         string numeroLicencia;
14
15     public:
16         bool validarHistorial();
17         bool registrarEvaluacion();
18
19     private:
20         EvaluacionVeterinaria * evaluacionesRegistradas;
21     };
22     #endif
```

Anexo E — Instrucciones detalladas de reproducibilidad

Para reproducir de forma íntegra el ciclo completo de desarrollo guiado por modelos (MDD) y la compilación del reporte técnico a partir de los artefactos del repositorio oficial:

1. **Clonar el repositorio institucional:** Abrir una terminal de comandos y ejecutar la descarga del repositorio git mediante la instrucción:
`git clone https://github.com/andymendozap03/PFC-MundiPets-MDD-EquipoH-BOUML.git`
2. **Cargar el proyecto estructural en el entorno CASE:** Iniciar la herramienta BOUML (versión 7.11 patch 4 o superior) y abrir el archivo maestro de configuración del metamodelo situado en la ruta:
`modelo/modelo.prj`
3. **Ejecutar la transformación automática Modelo-a-Texto (M2T):** Dentro del explorador de proyectos de BOUML, hacer clic derecho sobre el paquete raíz *modelo*, navegar hacia el menú interactivo *Generate* y seleccionar la opción *C++*. El sistema inyectará automáticamente los dieciséis archivos de cabecera e implementación en el directorio físico *generado/*, aplicando las directivas parametrizadas y resguardando los bloques protegidos.
4. **Compilar la unidad demostrable del backend:** Acceder mediante la consola de comandos a la carpeta *generado/* y ejecutar el compilador nativo GCC (g++) para enlazar las clases y el punto de entrada principal mediante la instrucción:
`g++ Usuario.cpp Propietario.cpp Veterinario.cpp Mascota.cpp HistorialMedico.cpp Vacuna.cpp MascotaVacuna.cpp EvaluacionVeterinaria.cpp main.cpp -o mundipets_demo`
5. **Ejecutar la unidad demostrable:** Una vez finalizada la compilación de forma exitosa sin advertencias, inicializar el archivo ejecutable resultante en el entorno local para validar la navegación de asociaciones en consola mediante la instrucción:
`./mundipets_demo.exe`
6. **Compilar de forma nativa este informe técnico:** En caso de requerir la regeneración física de este documento PDF desde el código fuente, situarse en el directorio *docs/* y ejecutar secuencialmente las directivas de automatización:
`pdflatex informe.tex`
`biber informe`
`pdflatex informe.tex` (ejecutar dos veces adicionales para estabilizar las referencias cruzadas y los índices).